



RepairHub

*Web enfocada en la gestión de reparación de
dispositivos electrónicos.*



salesianos
ALCALÁ DE HENARES

Tutor: Víctor Ramos

Curso académico: 2025 / 2026

Titulación: CFGM en Sistemas Microinformáticos y Redes (SMR)

Centro educativo: Las Naves Salesianos - Alcalá de Henares

Autor: Roberto Carrascoso Jordán

RESUMEN

Este documento recoge el diseño, desarrollo, despliegue y pruebas de **RepairHub**, una página web orientada a la gestión de pequeños y medianos talleres dedicados a la reparación de dispositivos electrónicos (teléfonos móviles, tablets, ordenadores portátiles, ordenadores de sobremesa, consolas y otros equipos similares). La web digitaliza los pasos de una reparación, desde que llega el dispositivo hasta su entrega al cliente, facilitando tareas que en muchos talleres del sector se siguen realizando mediante cuadernos en papel, hojas de cálculo o pizarras. Entre las funciones implementadas en la web hay: gestión de clientes con detección de duplicados y autocompletado, generación automática de códigos únicos de reparación, generación de un resguardo PDF con etiqueta recortable, control de estados a través de una máquina de estados, gestión de presupuestos y precios finales, búsqueda global con respuesta en tiempo real, exportación de listados a *CSV*, panel de control con indicadores y gráficos, y un sistema de autenticación con dos roles diferenciados (administrador y técnico).

La web se ha desarrollado con: Python, utilizando Flask, base de datos MariaDB, HTML5, Jinja2, CSS y JavaScript, la librería Chart.js para los gráficos de la página principal. La generación de PDF se hace con ReportLab. La web final se ha desplegado en una segunda máquina virtual con Debian 13, ejecutando la aplicación con Gunicorn detrás de Nginx. Ambas máquinas virtuales operan en una red local en modo adaptador puente, reproduciendo un escenario real.

Repositorio del código: https://github.com/robertocarrascoso/TFGM_RepairHub

Declaración de uso de IA: durante el desarrollo de este proyecto se ha utilizado el asistente Claude Code (Anthropic) como apoyo en tareas de programación y revisión de errores.

ÍNDICE

3. INTRODUCCIÓN	6
3.1 Presentación del proyecto	6
3.2 Contexto en el que se desarrolla	6
3.3 Objetivos del trabajo	7
3.3.1 Objetivo general	7
3.3.2 Objetivos específicos	7
4. DESCRIPCIÓN DEL PROBLEMA Y CONTEXTO	8
4.1 Caracterización del sector	8
4.2 Descripción detallada del problema	8
4.3 Justificación de la necesidad	9
4.4 Destinatarios y partes interesadas	9
4.5 Alcance y limitaciones	9
5. METODOLOGÍA	10
5.1 Enfoque general: desarrollo iterativo incremental	10
5.2 Análisis preliminar y requisitos	10
5.3 Diseño previo a la implementación	10
5.4 Procedimiento de implementación	10
5.5 Procedimiento de despliegue y validación	10
5.6 Herramientas de gestión del proyecto	11
5.7 Tecnologías utilizadas	11
6. RESULTADOS ESPERADOS	12
6.1 Producto final esperado	12
6.2 Requisitos funcionales	12
6.3 Casos de uso principales	13
6.4 Indicadores de éxito	14
7. PLAN DE TRABAJO	15
7.1 Fases del proyecto	15
7.2 Diagrama de Gantt	15
8. RECURSOS NECESARIOS	16
8.1 Recursos *hardware*	16
8.2 Recursos *software*	16
8.3 Recursos humanos	16
9. ESTUDIO DE VIABILIDAD	17
9.1 Viabilidad técnica	17
9.2 Viabilidad económica	17
9.3 Viabilidad material	17
9.4 Viabilidad temporal	17
9.5 Análisis de riesgos	18
9.6 Conclusión del estudio de viabilidad	18

10. IMPLEMENTACIÓN Y DESARROLLO	19
10.1 Arquitectura general del sistema	19
10.1.1 Separación entre desarrollo y producción	19
10.1.2 Flujo de una petición en producción	19
10.1.3 Estructura de directorios del proyecto	19
10.2 Modelo de datos	20
10.2.1 Tabla `clientes`	20
10.2.2 Tabla `reparaciones`	21
10.2.3 Tablas `contadores`, `historial_estados` y `usuarios`	21
10.3 Capa de aplicación: backend en Flask	22
10.3.1 Estructura de `app.py`	22
10.3.2 Configuración separada del código	22
10.3.3 Listado completo de rutas	23
10.3.4 Patrón de cada ruta	23
10.3.5 Mensajes al usuario	24
10.4 Capa de presentación: plantillas Jinja2	24
10.5 Hoja de estilos y tema dual claro/oscuro	24
10.6 Lógica de cliente en JavaScript	25
10.7 Generación de documentos PDF	25
10.8 Máquina de estados de las reparaciones	26
10.8.1 Transiciones permitidas	26
10.8.2 Estados con datos adicionales	26
10.9 Sistema de autenticación y autorización	27
10.9.1 Decoradores de protección	27
10.9.2 Flujo de inicio de sesión	27
10.9.3 Almacenamiento seguro de contraseñas	27
10.9.4 Roles	27
10.10 Cronología detallada del desarrollo	28
10.11 Despliegue en producción	28
10.11.1 Creación de la máquina virtual servidor	28
10.11.2 Incidencia con HestiaCP	29
10.11.3 Componentes instalados	29
10.11.4 Creación de la base de datos en producción	29
10.11.5 Servicio de arranque automático	29
10.11.6 Configuración de Nginx	30
10.11.7 Resolución del nombre `repairhub.local`	30
10.11.8 Script de despliegue	30
10.12 Decisiones de diseño técnico	31
10.13 Capturas de la interfaz	31
11. ESTUDIO DE RESULTADOS	35
11.1 Plan de pruebas	35

11.2 Resultados por bloque funcional	35
11.3 Resultados frente a los indicadores definidos	36
11.4 Incidencias detectadas y resolución	36
11.5 Cumplimiento de objetivos	37
12. CONCLUSIONES	38
12.1 Síntesis de logros	38
12.2 Aprendizajes profesionales	38
12.3 Limitaciones del trabajo	38
12.4 Líneas futuras de mejora	38
12.5 Reflexión final	39
13. REFERENCIAS BIBLIOGRÁFICAS	40

3. INTRODUCCIÓN

3.1 Presentación del proyecto

RepairHub es una aplicación web que resuelve, de forma sencilla y gratuitamente, la gestión administrativa de un taller pequeño o mediano de reparación de dispositivos electrónicos. Sustituye los cuadernos en papel y las hojas de cálculo improvisadas todavía habituales en el sector por una base de datos centralizada, accesible desde cualquier dispositivo de la red local del establecimiento. Cubre el ciclo completo de una reparación: alta del cliente, registro del dispositivo, emisión del resguardo, control de estados, presupuesto y precio final, búsqueda, exportación de listados y panel de control con indicadores.

El proyecto integra los contenidos dados en el transcurso del Ciclo Formativo de Grado Medio en Sistemas Microinformáticos y Redes (SMR), sistemas operativos, configuración de servidores, redes locales, bases de datos, *scripting* y programación básica.

3.2 Contexto en el que se desarrolla

El trabajo se realizó en el módulo de **Proyecto Intermodular** del segundo curso del ciclo SMR, en Las Naves Salesianos (Alcalá de Henares), en el curso 2025/2026.

El sector donde cobra vida esta web son las **microempresas de servicios técnicos de reparación**: plantillas de personal pequeñas, bajo conocimiento de las últimas tecnologías y una gestión que todavía se apoya en papel y sin orden alguno.

3.3 Objetivos del trabajo

3.3.1 Objetivo general

Diseñar, implementar, desplegar y documentar una web funcional que sustituya la gestión manual de un taller de reparación de dispositivos electrónicos, con una interfaz sencilla, accesible desde cualquier navegador de la red local y autosuficiente (sin depender de servicios de terceros).

3.3.2 Objetivos específicos

- **OE1.** Diseñar un modelo de datos relacional con las entidades mínimas del dominio: clientes, reparaciones, historial de estados, contador anual de códigos y usuarios.
- **OE2.** Implementar el *backend* con Python y Flask, con al menos veinte rutas HTTP para el dar de alta, modificar, eliminar, buscar y consultar.
- **OE3.** Diseñar una interfaz minimalista y adaptativa, con dos temas claro/oscuro, sin CSS pesados ni librerías JavaScript de componentes.
- **OE4.** Generar automáticamente un PDF A4 que incluya, en una sola hoja, el resguardo del cliente y la etiqueta recortable del dispositivo, separados por una línea de corte.
- **OE5.** Implementar autenticación con dos roles (administrador y técnico), proteger las rutas, almacenar las contraseñas en *hash* y disponer de un panel de administración de usuarios.
- **OE6.** Desplegar la aplicación en una máquina virtual independiente, con MariaDB como base de datos, Nginx como “mediador” el cual recibe las peticiones de los usuarios y los dirige a la app, Gunicorn como servidor que ejecuta el código de la web y como arranque automático se usa configuración en *systemd*.
- **OE7.** Automatizar el despliegue con un *script* que sincronice los archivos modificados y reinicie el servicio en una sola petición.
- **OE8.** Verificar el sistema con un plan de pruebas funcionales que cubra todas las pruebas.
- **OE9.** Documentar el proceso completo, técnico y académico, en una memoria que respete el formato.

4. DESCRIPCIÓN DEL PROBLEMA Y CONTEXTO

4.1 Caracterización del sector

El sector de la reparación de dispositivos electrónicos está muy anticuado. La mayoría del mercado son pequeñas empresas familiares o autónomos que dan servicio a clientes particulares de su barrio o ciudad. El taller tipo al que se dirige RepairHub tiene este perfil:

- **Plantilla reducida**, normalmente de uno a cinco operarios.
- **Local físico** en planta baja, accesible al cliente.
- **Volumen medio-alto** de reparaciones diarias, cada una con poca facturación unitaria.
- **Catálogo amplio pero centrado** en pantallas, baterías, conectores de carga y recuperación de información.
- **Clientes recurrentes** que vuelven al mismo taller.
- **Procesos administrativos poco digitalizados**, basados en anotaciones manuales o cuadernos físicos.

4.2 Descripción detallada del problema

El recorrido de una reparación en un taller sin una herramienta, pasa por: recepción del dispositivo y anotación de datos, resguardo en papel, diagnóstico y presupuesto, comunicación del presupuesto, reparación una vez aceptado, aviso de recogida y por último la entrega y cobro.

Gestionarlo solo con papel genera varios problemas:

- **Búsqueda lenta:** localizar una reparación concreta te obliga a buscar a mano entre los cuadernos. Si el cliente llama y solo da su nombre, sin código, esa búsqueda puede consumir varios minutos en los que no se atiende a nadie más.
- **Riesgo de pérdida de información:** un cuaderno mojado o perder algún papel borra el registro de una reparación que el cliente sí conserva en su resguardo (o al revés).
- **Estadísticas imposibles:** preguntas básicas para la gestión "¿cuántas pantallas hemos hecho este mes?", "¿cuál es el precio medio?", "¿qué reparaciones llevan más de una semana paradas?" nadie las responde porque el coste manual supera al beneficio.

- **Sin historial por cliente:** cuando un cliente vuelve, no es inmediato consultar sus reparaciones anteriores, lo que dificulta detectar averías repetidas o aplicar garantías y descuentos.

4.3 Justificación de la necesidad

- **Valor para el taller.** Reduce el tiempo de búsqueda de cualquier registro, elimina errores de transcripción, emite resguardos y deja un registro de cada cambio de estado.
- **Coste competitivo.** Las soluciones *SaaS* del nicho cobran una cuota mensual por usuario y RepairHub es una herramienta auto alojada, eliminando ese problema.
- **Independencia.** Funciona en la red local aunque se caiga internet, y no depende de ningún proveedor.
- **Relación con SMR.** Integra Linux en servidor, base de datos, servidor web, *systemd*, redes y programación.

4.4 Destinatarios y partes interesadas

Los destinatarios directos son los **operarios de talleres pequeños y medianos**. Las partes interesadas son:

- **Personal técnico:** usuario diario. Necesita rapidez, claridad y pocos pasos por operación.
- **Propietario o personal administrativo:** usuario interesado en indicadores y trazabilidad. Necesita panel de control y exportación de listados.
- **Cliente final del taller:** beneficiario indirecto porque recibe un resguardo profesional y respuestas rápidas.

4.5 Alcance y limitaciones

El alcance cubre todo lo que va **desde la recepción del dispositivo hasta su entrega**, más la gestión de clientes y las estadísticas internas.

5. METODOLOGÍA

5.1 Enfoque general: desarrollo iterativo incremental

He dividido el trabajo en ocho fases. Cada fase persigue un objetivo concreto y deja la web en estado funcional.

5.2 Análisis preliminar y requisitos

La captura de requisitos se basó en la experiencia propia y el análisis a soluciones similares (RepairShopr y RepairDesk).

5.3 Diseño previo a la implementación

Antes de escribir código preparé:

- Los estados de una reparación, con los estados posibles y las transiciones válidas.
- El logotipo y el esquema visual en Figma.
- El reparto de tamaño del PDF en el resguardo del cliente.
- Qué datos queríamos recoger por cliente, reparaciones y gestión de estos, para así tener una idea clara de qué y cómo hacer la base de datos.

5.4 Procedimiento de implementación

Cada fase sigue el mismo ciclo: definir el objetivo y los criterios, escribir el código, probarlo a mano en el navegador, hacer un commit descriptivo en Git y sincronizar con el repositorio de GitHub.

5.5 Procedimiento de despliegue y validación

El despliegue se hace en una segunda máquina virtual con Debian 13 sin entorno gráfico, sobre la que se instalan MariaDB, Nginx y Python con Gunicorn y se configura el arranque automático con *systemd*. Un *script* sincroniza los archivos entre máquinas y reinicia la web.

5.6 Herramientas de gestión del proyecto

Tabla 1. *Software utilizado en la máquina virtual de desarrollo.*

Software	Utilidad en el proyecto
Git + GitHub	Control de versiones local y repositorio remoto.
VS Code	Editor de código.
Python 3 + venv	Entorno de ejecución del backend.
Figma	Diseño del logotipo e idea inicial visual.

5.7 Tecnologías utilizadas

Software	Tecnología
SO desarrollo	Ubuntu
SO producción	Debian
Lenguaje backend	Python
Framework web	Flask
Conector BD	mysql-connector-python
Generador PDF	ReportLab
Servidor WSGI	Gunicorn
Sistema gestor BD	MariaDB
Proxy inverso	Nginx
Cliente	HTML5 + CSS3 + JavaScript <i>vanilla</i>
Gráficos	Chart.js
Virtualización	VirtualBox
Editor y estados	VS Code, Git + GitHub

6. RESULTADOS ESPERADOS

6.1 Producto final esperado

Una aplicación web operativa, desplegada en una MV Debian 13 y accesible desde cualquier navegador de la red local, que cubre el ciclo completo de un taller: autenticación con dos roles, panel de control con estadísticas y gráficas, generación de PDF, búsqueda global y exportación de listados.

6.2 Requisitos funcionales

Describen **qué** debe hacer el sistema.

Tabla 2. *Requisitos funcionales del sistema.*

ID	Requisito
RF1.1	Permitir el alta de un cliente con nombre y al menos un dato de contacto (teléfono o correo).
RF1.2	Detectar clientes existentes por teléfono o correo para evitar duplicados.
RF1.3	Consultar el historial completo de reparaciones de cualquier cliente.
RF1.4	Editar los datos identificativos y de contacto de un cliente.
RF1.5	Calcular el gasto total acumulado por cliente sumando el precio final de las reparaciones entregadas.
RF2.1	Crear una reparación asociada a un cliente, con tipo de dispositivo, marca, modelo, avería, observaciones y técnico responsable.
RF2.2	Generar un código único REP-AAAA-NNNNN, con contador independiente por año.
RF2.3	Consultar la ficha completa de una reparación a partir de su código.
RF2.4	Cambiar el estado de una reparación solo entre transiciones permitidas.
RF2.5	Registrar en un historial cada cambio de estado, con técnico, fecha y hora.
RF2.6	Editar los datos del dispositivo y la avería tras crear la reparación.
RF2.7	Eliminar una reparación previa confirmación.
RF2.8	Asociar un presupuesto al pasar a "Presupuesto enviado" y registrar la respuesta del cliente (aceptado o rechazado).
RF2.9	Fijar un precio final, que puede coincidir o no con el presupuesto.
RF3.1	Generar al crear la reparación un PDF A4 con el resguardo arriba y la etiqueta recortable abajo, separados por una línea de corte.
RF3.2	Imprimir el PDF en cualquier momento a partir del código de reparación.
RF4.1	Buscado global que filtre a la vez reparaciones (por código o avería) y clientes (por nombre, teléfono o correo).
RF4.2	Actualizar los resultados en tiempo real conforme se escribe, sin refrescar.
RF5.1	Mostrar al iniciar sesión un panel con: reparaciones pendientes, reparaciones del mes, ingresos del mes y tiempo medio de reparación.
RF5.2	Gráfico de barras con el conteo de reparaciones por estado.
RF5.3	Gráfico <i>donut</i> con la distribución por tipo de dispositivo.

RF5.4	Listar las últimas cinco reparaciones con enlace a su detalle.
RF6.1	Proteger todas las rutas con autenticación, salvo el inicio de sesión.
RF6.2	Dos roles diferenciados: administrador y técnico.
RF6.3	Restringir el panel de administración solo a administradores.
RF6.4	Almacenar las contraseñas en <i>hash</i> , nunca sin hashear.
RF6.5	Impedir que un administrador se elimine a sí mismo.
RF7.1	Filtrar el listado de reparaciones por estado, rango de fechas, tipo de dispositivo y cliente.
RF7.2	Paginar el listado de reparaciones a veinte por página.
RF7.3	Paginar el listado de clientes a veinte por página.
RF7.4	Exportar el listado de reparaciones filtrado a CSV.
RF8.1	Dos temas visuales (claro y oscuro) seleccionables y persistentes entre sesiones.
RF8.2	Funcionar bien en móvil, tableta y escritorio.
RF8.3	Resaltar el enlace activo del menú de navegación.
RF8.4	Mostrar junto a "Reparaciones" un contador de pendientes.

6.3 Casos de uso principales

Tabla 3. *Casos de uso principales.*

Caso de uso	Actor	Resumen del flujo principal
Dar entrada a un dispositivo nuevo	Técnico	Abrir el formulario, introducir datos de cliente, dispositivo y enviar. El sistema crea el cliente si no existía, genera el código y abre el PDF.
Consultar el estado de una reparación	Técnico	Localizar la reparación por código o búsqueda, abrir la ficha y revisar el historial.
Cambiar el estado de una reparación	Técnico	Desde la ficha, elegir el siguiente estado válido, si hace falta un dato (presupuesto o precio final), aportarlo.
Buscar un cliente recurrente	Técnico	Al rellenar la nueva entrada, el sistema sugiere coincidencias por teléfono y autorrellena nombre y correo.
Imprimir o reimprimir el resguardo	Técnico	Desde la ficha, pulsar "Imprimir PDF", se abre en pestaña nueva.
Crear un nuevo técnico	Administrador	Desde el panel de administración, introducir nombre, correo, contraseña y rol, se rechaza si el correo ya existe.
Consultar estadísticas del mes	Cualquiera	Al iniciar sesión, aterrizar en el panel con indicadores y dos gráficos.
Exportar el listado a CSV	Técnico	Aplicar filtros, pulsar "Exportar CSV" y descargar el archivo.

6.4 Indicadores de éxito

Indicadores para evaluar el cumplimiento de los objetivos.

Tabla 4. *Indicadores de éxito y métricas asociadas.*

Indicador	Métrica	Mínimo aceptable
Funcionalidad implementada	Número de rutas HTTP funcionales	≥ 20
Cobertura del modelo de datos	Número de tablas relacionadas	5
Disponibilidad en producción	La aplicación se sirve tras un reinicio del servidor	Sí
Autenticación y autorización	Las rutas protegidas rechazan el acceso sin sesión	Sí (todas)
Seguridad de contraseñas	Las contraseñas no aparecen en claro en la base de datos	Sí (todas)
Generación de PDF	El PDF se imprime correctamente desde un navegador estándar	Sí
Adaptación responsiva	La web se ve bien en cualquier pantalla	Sí
Tiempo de respuesta de la búsqueda	Latencia baja en red local	Inmediata
Procedimiento de despliegue	Un <i>script</i> aplica los cambios y reinicia el servicio	Sí
Documentación	La memoria incluye los 13 apartados oficiales y el anexo de código	Sí

7. PLAN DE TRABAJO

7.1 Fases del proyecto

El desarrollo se hizo en **8 fases**, ejecutadas del **16 de marzo hasta el 9 de mayo de 2026**.

Tabla 5. *Fases del proyecto, calendario y contenido principal.*

Fase	Periodo	Contenido principal
F1	16 – 22 marzo	Preparación del entorno (Python, venv, Git) y diseño y creación de la base de datos.
F2	23 – 29 marzo	Backend Flask inicial, plantilla base y primera versión del panel de control.
F3	30 marzo – 6 abril	Estilos CSS, tema dual claro/oscuro, JavaScript y formulario de nueva entrada con autocompletado.
F4	7 – 13 abril	Generación de PDF con ReportLab y gestión de reparaciones (listado, ficha, estados, presupuestos).
F5	14 – 20 abril	Gestión de clientes, búsqueda global con AJAX y panel de control con Chart.js.
F6	21 – 27 abril	Script <code>seed.py</code> con datos de prueba, primer ciclo de pruebas y rediseño visual.
F7	28 abril – 4 mayo	Mejoras pre-producción: autenticación con dos roles, panel de administración, paginación, filtros y exportación CSV.
F8	5 – 9 mayo	Despliegue en la MV Debian 13 (MariaDB, Nginx, Unicorn, <i>systemd</i>) y validación final.

7.2 Diagrama de Gantt

La celda marcada indica la semana en que se trabaja cada fase.

Tarea	S1	S2	S3	S4	S5	S6	S7	S8
F1 — Entorno y base de datos	■							
F2 — Backend Flask y panel inicial		■						
F3 — Estilos, tema y nueva entrada			■					
F4 — PDF y gestión de reparaciones				■				
F5 — Clientes, búsqueda y panel					■			
F6 — Datos de prueba y rediseño						■		
F7 — Autenticación y mejoras							■	
F8 — Despliegue y validación								■
Redacción de la memoria	■	■	■	■	■	■	■	■

8. RECURSOS NECESARIOS

Este apartado lista lo que ha hecho falta para sacar adelante el proyecto: equipos, programas y trabajo de personas. Todo el coste real ha sido cero, porque se ha usado software libre y máquinas virtuales.

8.1 Recursos **hardware**

No se ha comprado nada. Todo corre sobre un ordenador con VirtualBox, dentro del cual se han creado dos máquinas virtuales. La máquina de desarrollo es un Ubuntu 24.04 LTS con escritorio, con 8 GB de RAM y 40 GB de disco, y es donde se prueba el código. La máquina de producción es un Debian 13 sin escritorio, con 4 GB de RAM y 40 GB de disco, y es donde se despliega la web final. Las dos usan el adaptador de red en modo puente, así que cada una recibe su propia IP en la red local y se comportan como dos ordenadores físicos distintos en el taller.

8.2 Recursos **software**

Todo el *software* es libre y gratuito, así que el presupuesto es de 0 €. Los sistemas operativos son Ubuntu (desarrollo) y Debian 13 (producción). El backend usa Python 3.13 con Flask y sus librerías (Jinja2 para las plantillas, Werkzeug para la seguridad de contraseñas, mysql-connector-python para hablar con la base de datos, ReportLab para el PDF y python-dotenv para leer la configuración). La base de datos es MariaDB. En el servidor se usan Nginx para redirigir tráfico y recibir datos y Gunicorn para ejecutar la web. La parte de cliente es HTML, CSS y JavaScript, con Chart.js para los gráficos del panel y la tipografía Inter. Las herramientas de trabajo son VirtualBox, VS Code, Git, GitHub y Figma.

8.3 Recursos humanos

El proyecto lo ha desarrollado una sola persona.

9. ESTUDIO DE VIABILIDAD

9.1 Viabilidad técnica

Las tecnologías elegidas (Python, Flask, MariaDB, Nginx, Gunicorn) están muy documentadas, lo que facilita resolver los problemas que van apareciendo. La web está pensada para que la pueda terminar una persona en el tiempo disponible.

9.2 Viabilidad económica

El coste del proyecto ha sido nulo: todo el software es libre y gratuito. Una herramienta autoalojada como esta evita pagar cuotas mensuales, cuyo coste no termina nunca, por lo que resulta sostenible para un taller pequeño.

9.3 Viabilidad material

No hacen falta ordenadores de alto rendimiento: la máquina Debian 13 funciona en prácticamente todo tipo de ordenadores, sin necesidad de comprar uno nuevo.

9.4 Viabilidad temporal

El trabajo se planificó en ocho fases de una semana, entre el 16 de marzo y el 9 de mayo de 2026, con la memoria redactada en paralelo.

9.5 Análisis de riesgos

Se identificaron desde el principio los riesgos técnicos y de seguridad que podían comprometer el proyecto. La Tabla 6 los recoge.

Tabla 6. *Análisis de riesgos del proyecto.*

Riesgo	Probabilidad	Impacto	Mitigación
Subestimar el tiempo de desarrollo	Media	Alto	Fases cortas y revisión de avance cada dos semanas.
Error técnico imprevisto	Media	Medio	Investigar antes de cada parte y tener una alternativa prevista.
Pérdida de código	Baja	Muy alto	Repositorio de Git en GitHub.
Fallo de red entre las dos máquinas virtuales	Media	Medio	Comprobar la conexión después de cada cambio de red.
Filtrado de credenciales	Baja	Alto	Archivo de credenciales excluido del control de versiones.
Falta de tiempo para la memoria	Media	Alto	Documentar en paralelo al desarrollo, no al final.
Diferencias entre desarrollo y producción	Media	Alto	Retirar el modo de datos de prueba y validar en servidor real.

9.6 Conclusión del estudio de viabilidad

El proyecto es viable en todos los aspectos (técnico, económico, material y temporal), es una alternativa gratuita y modificable a gusto personal.

10. IMPLEMENTACIÓN Y DESARROLLO

Cómo está hecha la web por dentro y qué decisiones se tomaron. Se explica primero la arquitectura general, luego el modelo de datos, cada sección de la web y por último el despliegue real en el servidor.

10.1 Arquitectura general del sistema

10.1.1 Separación entre desarrollo y producción

El sistema se monta sobre dos máquinas virtuales VirtualBox dentro de la misma red local. La de desarrollo (Ubuntu, IP 192.168.1.140) tiene VS Code, Python, Git y el navegador; es donde se programa y se prueba. La de producción (Debian 13, IP 192.168.1.143) tiene Python 3.13.5, MariaDB 11.8.6, Nginx 1.26.3 y Gunicorn corriendo como servicio; es donde se publica la web, accesible desde la red local. La comunicación entre ambas es por SSH y la sincronización de archivos se hace con la orden `rsync`, que solo transfiere las diferencias entre origen y destino en lugar de copiarlo todo. Solo se envía a producción el código que ya se ha comprobado que funciona en desarrollo.

10.1.2 Flujo de una petición en producción

Cuando un técnico abre la web, la petición pasa por varias capas. Primero llega a Nginx, que escucha en el puerto 80 si se pide un archivo estático (CSS, JavaScript o imágenes) lo entrega él mismo directamente, para todo lo demás actúa de *proxy* inverso y reenvía la petición a Gunicorn. Gunicorn, en el puerto interno 5000, ejecuta la aplicación Flask repartiendo el trabajo entre dos procesos. Flask procesa la lógica y, cuando necesita datos, consulta MariaDB. La respuesta vuelve por el mismo camino hasta el navegador. Es el montaje estándar para publicar una aplicación Flask: Nginx da la cara y descarga de trabajo estático, Gunicorn ejecuta la aplicación y Flask que habla con la base de datos.

10.1.3 Estructura de directorios del proyecto

El proyecto se organiza en una carpeta raíz con el archivo principal `app.py` (todo el *backend*), `config.py` (configuración), `requirements.txt` (dependencias) y un archivo de credenciales que queda fuera de Git. A partir de ahí hay cuatro carpetas: `base-de-datos`, con el *script* de creación de las tablas y el de datos de prueba; `estaticos`, con la hoja de

estilos, el JavaScript y las imágenes; **plantillas**, con las páginas HTML (la base común, el inicio de sesión, el panel, los formularios y los listados); y **utilidades**, con el generador de PDF. El *script* de despliegue vive aparte, en la máquina de desarrollo y fuera del repositorio, porque contiene credenciales del servidor.

10.2 Modelo de datos

El modelo tiene **cinco tablas**. La tabla `clientes` guarda los datos de contacto. La tabla `reparaciones` es la principal y apunta al cliente al que pertenece cada reparación. La tabla `historial_estados` registra cada cambio de estado de una reparación. La tabla `contadores` lleva el número correlativo de códigos por año. Y la tabla `usuarios` guarda las cuentas que pueden iniciar sesión. Las relaciones son: cada reparación pertenece a un cliente, y cada fila del historial pertenece a una reparación.

10.2.1 Tabla `'clientes'`

Tabla 7. Tabla `'clientes'` y descripción de columnas.

Columna	Tipo	Descripción
id	INT AUTO_INCREMENT PRIMARY KEY	Identificador interno.
nombre	VARCHAR(100) NOT NULL	Nombre del cliente. Obligatorio.
telefono	VARCHAR(20)	Opcional. Se exige al menos uno entre teléfono y correo.
email	VARCHAR(100)	Opcional.
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	Fecha de alta.

La regla "al menos uno de los dos contactos" se comprueba en el código de la aplicación, porque MariaDB no admite una restricción sencilla para algo así.

10.2.2 Tabla `reparaciones`

Tabla 8. Tabla `reparaciones` y descripción de columnas.

Columna	Tipo	Descripción
id	INT AUTO_INCREMENT PRIMARY KEY	Identificador interno.
codigo	VARCHAR(20) NOT NULL UNIQUE	Código legible (REP-AAAA-NNNNN), único.
cliente_id	INT NOT NULL, FK → clientes.id	Cliente al que pertenece.
tipo_dispositivo	VARCHAR(50) NOT NULL	Móvil, tableta, portátil, etc.
marca	VARCHAR(50)	Marca del dispositivo.
modelo	VARCHAR(50)	Modelo.
averia	TEXT NOT NULL	Descripción del problema.
observaciones	TEXT	Notas adicionales del técnico.
estado	VARCHAR(30) DEFAULT 'Recibido'	Estado en la máquina de estados.
presupuesto	DECIMAL(8,2)	Importe presupuestado.
precio_final	DECIMAL(8,2)	Importe finalmente cobrado.
presupuesto_aceptado	BOOLEAN DEFAULT NULL	NULL, TRUE (aceptado) o FALSE (rechazado).
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	Fecha de creación.
updated_at	DATETIME ON UPDATE CURRENT_TIMESTAMP	Última modificación.

El campo `updated_at` se actualiza solo cada vez que cambia la fila, lo que permite calcular el tiempo medio de reparación restando la fecha de creación de la de última modificación.

10.2.3 Tablas `contadores`, `historial\_estados` y `usuarios`

Tabla 9. Tablas `contadores`, `historial\_estados` y `usuarios`.

Tabla	Función	Estructura resumida
contadores	Contador anual de códigos REP-AAAA-NNNNN. Una fila por año.	year (PK), ultimo_num.
historial_estados	Guarda cada cambio de estado. Sirve de auditoría.	id (PK), reparacion_id (FK), estado, tecnico, fecha.
usuarios	Cuentas que pueden iniciar sesión. Dos roles: admin y tecnico.	id, nombre, email (único), password_hash, rol, created_at.

El nombre del técnico se guarda en `historial_estados` como texto libre, no como referencia a la tabla `usuarios`, para que el historial no se rompa si más adelante se borra ese usuario. Toda la base de datos usa codificación `utf8mb4`, así que admite tildes, eñes y emojis sin problemas.

10.3 Capa de aplicación: backend en Flask

10.3.1 Estructura de `app.py`

El backend es un único archivo `app.py` de 759 líneas, ordenado en bloques: las importaciones, la creación de la aplicación Flask, la seguridad, la función que abre la conexión a la base de datos, funciones para filtros y para contar reparaciones pendientes, el diccionario que define las transiciones de estado y las 22 rutas. Para un proyecto de este tamaño no compensa partir el archivo en módulos separados porque añadiría complejidad sin ganar nada.

10.3.2 Configuración separada del código

Los datos sensibles (usuario y contraseña de la base de datos y la clave secreta de las sesiones) no están escritos dentro del código. Se guardan en un archivo de entorno aparte que está excluido de Git, y `config.py` los lee al arrancar usando `python-dotenv`; si una variable no existe, usa un valor por defecto pensado para desarrollo. La clave secreta de producción se genera directamente en el servidor con una orden que produce una cadena aleatoria larga. Así las credenciales nunca aparecen en el repositorio y el mismo código sirve para los dos entornos cambiando solo ese archivo.

10.3.3 Listado completo de rutas

Tabla 10. Listado completo de rutas implementadas.

Método	Ruta	Descripción
GET / POST	/login	Formulario y autenticación
GET	/logout	Cierre de sesión
GET	/	Panel de control
GET / POST	/nueva-entrada	Crear nueva reparación
GET	/reparaciones	Listado con filtros y paginación
GET	/reparaciones/csv	Exportación a CSV
GET	/reparacion/{codigo}	Detalle de una reparación
GET / POST	/reparacion/{codigo}/editar	Editar datos del dispositivo
POST	/reparacion/{codigo}/eliminar	Eliminar reparación
POST	/reparacion/{codigo}/cambiar-estado	Cambiar estado
POST	/reparacion/{codigo}/presupuesto	Asignar presupuesto
POST	/reparacion/{codigo}/precio-final	Asignar precio final
GET	/clientes	Listado con paginación
GET	/cliente/{id}	Detalle de un cliente
GET / POST	/cliente/{id}/editar	Editar datos del cliente
GET	/buscar	Página de búsqueda
GET	/api/buscar	Servicio JSON de búsqueda
GET	/api/buscar-cliente	Servicio JSON de autocompletado
GET	/pdf/{codigo}	Servir el PDF en línea
GET	/admin	Panel de administración (solo admin)
POST	/admin/crear-usuario	Crear usuario (solo admin)
POST	/admin/eliminar-usuario/{id}	Eliminar usuario (solo admin)

En total son 22 rutas.

10.3.4 Patrón de cada ruta

Casi todas las rutas siguen el mismo guión: aplican protección, abren la conexión con la base de datos, validan los parámetros, ejecutan las consultas SQL siempre con parámetros (es decir, los valores no se pegan dentro del texto de la consulta sino que se pasan aparte, lo que evita la inyección SQL), confirman los cambios si han modificado datos, cierran la conexión y terminan mostrando un mensaje al usuario y redirigiendo o pintando una página.

10.3.5 Mensajes al usuario

El resultado de cada operación se le comunica al usuario con el mecanismo propio de Flask para mensajes temporales, con tres niveles (éxito, error e información) que la plantilla base pinta con su color correspondiente.

10.4 Capa de presentación: plantillas Jinja2

Las plantillas están en la carpeta `plantillas` y todas heredan de una plantilla base que aporta lo común: la cabecera, el menú de navegación, el contenedor principal y el pie. Los trozos que se repiten en varias páginas, como el *badge* de color que indica el estado de una reparación, se definen una sola vez como bloque reutilizable y se invocan donde hagan falta, para no repetir el mismo HTML una y otra vez.

10.5 Hoja de estilos y tema dual claro/oscuro

El diseño busca claridad antes que adornos: paleta reducida, una sola tipografía (Inter) y nada de animaciones llamativas. Se empezó con una paleta solo en blanco, negro y grises; en la fase 6 se añadió el azul como color de acción y se reservaron colores concretos para los bloques de estado. El cambio entre tema claro y oscuro se hace con variables CSS, en lugar de fijar los colores directamente, cada color se define una vez como variable y todo el resto de la hoja la usa, de modo que al cambiar un único atributo en la página se redibuja todo con la otra paleta. La preferencia del usuario se guarda en el navegador, así que el tema elegido se mantiene al recargar o al volver otro día. La Tabla 11 recoge los colores de cada tema.

Tabla 11. Tokens CSS del tema dual claro/oscuro.

Token	Tema claro	Tema oscuro	Función
--accent	#2563EB	#3B82F6	Color de acción principal
--accent-hover	#1E40AF	#60A5FA	Hover de botones azules
--danger	#DC2626	#EF4444	Acciones destructivas
--success	#16A34A	#22C55E	Confirmaciones
--warning	#D97706	#F59E0B	Avisos
--bg-primary	#FFFFFF	#0F172A	Fondo principal
--bg-secondary	#F8FAFC	#1E293B	Fondo de tarjetas
--bg-tertiary	#F1F5F9	#334155	Fondos atenuados
--text-primary	#0F172A	#F1F5F9	Texto principal
--text-secondary	#475569	#CBD5E1	Texto secundario
--text-muted	#94A3B8	#64748B	Texto atenuado
--border	#E2E8F0	#334155	Bordes

10.6 Lógica de cliente en JavaScript

El archivo `main.js` tiene toda la lógica de cliente, el cambio de tema guardado en el navegador, el menú *hamburguesa* en móvil, el autocompletado de clientes en la nueva entrada (consulta a la función de búsqueda de clientes con una pequeña espera de 300 ms tras dejar de teclear, para no lanzar una petición por cada letra), la búsqueda en tiempo real y la ventana emergente del PDF que aparece tras crear una reparación.

10.7 Generación de documentos PDF

El PDF se genera con la librería ReportLab. Es un único folio A4 partido en dos zonas separadas por una línea de corte discontinua. La parte de arriba es el resguardo para el cliente: lleva la cabecera con el nombre del taller, el código y la fecha de la reparación, los datos del cliente, los datos del dispositivo, la descripción de la avería y las observaciones, las condiciones del servicio en letra pequeña y una línea para la firma. La parte de abajo es una etiqueta más pequeña, enmarcada, pensada para recortarla y pegarla en el dispositivo. El módulo `pdf_generator.py` es una función que recibe la reparación y el cliente, dibuja todo eso sobre el folio y guarda el archivo en disco. Guardarlo en disco en lugar de generarlo solo en memoria hace que no se pierda fácilmente y permite reimprimir el mismo PDF más tarde sin volver a generarlo.

10.8 Máquina de estados de las reparaciones

El estado de una reparación no puede cambiar de cualquier manera: tiene un orden de estados con un recorrido fijo. La reparación entra en "Recibido", pasa a "Diagnosticado" y de ahí a "Presupuesto enviado". En este punto hay dos caminos: si el cliente acepta, pasa a "Presupuesto aceptado" y de ahí a "Reparando", si lo rechaza, salta directamente a "Entregado" y el aparato se devuelve sin tocar. Mientras está "Reparando" puede ir y volver entre ese estado y "Esperando pieza" tantas veces como haga falta. Cuando termina pasa a "Listo" y, al recogerlo el cliente, a "Entregado", que es el estado final.

10.8.1 Transiciones permitidas

La Tabla 12 indica, para cada estado actual, a qué estados se puede pasar.

Tabla 12. *Transiciones permitidas entre estados.*

Estado actual	Siguientes permitidos
Recibido	Diagnosticado
Diagnosticado	Presupuesto enviado
Presupuesto enviado	Presupuesto aceptado, Entregado (rechazo)
Presupuesto aceptado	Reparando
Reparando	Esperando pieza, Listo
Esperando pieza	Reparando
Listo	Entregado
Entregado	(estado final)

10.8.2 Estados con datos adicionales

Algunas transiciones necesitan información extra antes de poder ejecutarse. Para pasar a "Presupuesto enviado" se abre un formulario para meter el importe, y sin importe no se permite el cambio. Para pasar a "Listo" se pide el precio final, que por defecto se sugiere igual al presupuesto pero el técnico puede cambiar si hubo complicaciones. El rechazo del presupuesto no pide datos, pero marca la reparación como presupuesto no aceptado y el dispositivo se devuelve sin reparar.

10.9 Sistema de autenticación y autorización

10.9.1 Decoradores de protección

Hay dos decoradores que envuelven las rutas para añadir una comprobación antes de ejecutarlas. Un decorador es una función que "envuelve" a otra para añadirle comportamiento sin tocar su código. El primero exige tener sesión iniciada: si no la hay, redirige al inicio de sesión. El segundo exige además ser administrador: si el usuario tiene sesión pero su rol no es administrador, lo manda al panel con un aviso de acceso restringido. Así, basta con poner el decorador encima de una ruta para que quede protegida, sin repetir la comprobación dentro de cada función.

10.9.2 Flujo de inicio de sesión

Si no hay sesión, el decorador de protección redirige a la página de inicio de sesión. Allí el servidor busca el correo en la tabla de usuarios y compara la contraseña introducida con la guardada. Si coincide, guarda en la sesión el identificador, el nombre y el rol del usuario (la sesión va firmada con la clave secreta para que no se pueda falsificar). Si las credenciales son incorrectas, se muestra siempre el mismo mensaje genérico, para que nadie pueda averiguar qué correos existen probando uno a uno.

10.9.3 Almacenamiento seguro de contraseñas

Las contraseñas no se guardan nunca tal cual. Se almacenan transformadas con una función de Werkzeug que aplica un *hash* con sal: el *hash* es una huella irreversible de la contraseña, y la sal es un valor aleatorio distinto por usuario que se mezcla antes de calcularla. Gracias a la sal, dos usuarios con la misma contraseña tienen huellas distintas, y recuperar la contraseña original a partir de la huella es inviable en la práctica.

10.9.4 Roles

Hay dos roles. El administrador tiene acceso total, incluido el panel de administración de usuarios. El técnico puede hacer todas las operaciones del taller pero no entra en la administración. La plantilla base mira el rol del usuario activo para enseñar u ocultar el enlace al panel de administración.

10.10 Cronología detallada del desarrollo

Dentro de las ocho fases del apartado 7, el trabajo se fue haciendo en iteraciones técnicas, cada una con un objetivo concreto que se podía comprobar en el navegador.

Fase	Iteración técnica
F1	Estructura del proyecto, entorno virtual, dependencias y Git inicial.
F1	Creación de las tablas, instalación de MariaDB y creación del usuario de base de datos.
F2	Configuración con variables de entorno, aplicación Flask con la primera ruta del panel y plantillas base y de panel.
F3	Hoja de estilos con variables, tipografía Inter y componentes; diseño responsive en dos puntos de ruptura; JavaScript con cambio de tema y menú hamburguesa.
F3	Ruta de nueva entrada, validación, gestión del cliente con autocompletado y generación del código de reparación.
F4	Módulo de generación de PDF con ReportLab (resguardo y etiqueta) y ruta para servirlo.
F4	Listado de reparaciones con filtros y paginación, ficha con historial y cambios de estado, edición, eliminación, presupuesto y precio final.
F5	Listado de clientes y ficha con historial y gasto acumulado.
F5	Página y servicio de búsqueda con actualización en tiempo real.
F5	Panel de control: cuatro indicadores, gráfico de barras por estado y gráfico circular por tipo de dispositivo, últimas cinco reparaciones.
F6	Datos de prueba con dos usuarios, doce clientes y veinticinco reparaciones repartidas por la máquina de estados.
F6	Primer ciclo de pruebas, corrección de fallos y rediseño visual.
F7	Autenticación con usuarios, decoradores, panel de administración, paginación, filtros avanzados y exportación CSV.
F8	Instalación del <i>stack</i> en producción, servicio de arranque automático, Nginx como <i>proxy</i> inverso, resolución del nombre local, limpieza del modo de datos de prueba (el archivo principal pasó de 1132 a 759 líneas) y <i>script</i> de despliegue.

10.11 Despliegue en producción

10.11.1 Creación de la máquina virtual servidor

Sobre VirtualBox se creó una segunda máquina virtual con Debian 13. El adaptador de red en modo puente le da una IP propia en la red local, la 192.168.1.143.

10.11.2 Incidencia con HestiaCP

El plan inicial era usar HestiaCP, un panel que automatiza la configuración del servidor. Al ir a instalarlo se vio que su instalador oficial no soporta Debian 13 (solo Debian 11 y 12 y Ubuntu 22.04 y 24.04). Había dos opciones: bajar a Debian 12 o montar el servidor a mano. Se decidió mantener Debian 13 e instalar todo manualmente.

10.11.3 Componentes instalados

En el servidor se instalaron, mediante el gestor de paquetes de Debian, el sistema gestor de base de datos MariaDB con su cliente, el servidor web Nginx, Python 3 con soporte para entornos virtuales, y las herramientas Git y rsync para el despliegue. Las versiones finales fueron MariaDB 11.8.6, Nginx 1.26.3 y Python 3.13.5.

10.11.4 Creación de la base de datos en producción

En la base de datos del servidor se creó la base `repairhub` con codificación `utf8mb4`, un usuario propio para la aplicación con su contraseña y los permisos necesarios solo sobre esa base, y se recargaron los privilegios. La aplicación se conecta siempre con ese usuario limitado, nunca con el administrador de la base de datos.

10.11.5 Servicio de arranque automático

Para que la web arranque sola cuando se enciende el servidor y se recupere si se cae, Gunicorn se registra como un servicio del sistema mediante *systemd*. La definición del servicio indica con qué usuario debe ejecutarse, en qué carpeta, de dónde leer las variables de entorno, la orden de arranque (Gunicorn con dos procesos escuchando en el puerto interno 5000) y la instrucción de reiniciarse automáticamente si falla. El servicio se marca para arrancar con el sistema, de modo que tras un reinicio del servidor la web vuelve a estar disponible sin intervención.

10.11.6 Configuración de Nginx

Nginx se configura para escuchar en el puerto 80 y atender las peticiones dirigidas a `repairhub.local` o a la IP del servidor. Se le indica que los archivos estáticos los sirva él mismo desde la carpeta del proyecto (con caché de una semana) y que todo lo demás lo reenvíe a la aplicación Flask que corre en el puerto interno 5000, pasándole las cabeceras necesarias para que la aplicación sepa la dirección real del cliente.

10.11.7 Resolución del nombre `repairhub.local`

En lugar de montar un servidor DNS interno, se añade una línea en el archivo de equivalencias de nombres de cada ordenador que vaya a usar la web, asociando el nombre `repairhub.local` a la IP del servidor (192.168.1.143). Así los técnicos escriben un nombre fácil en vez de la dirección numérica.

10.11.8 Script de despliegue

En la máquina de desarrollo hay un *script* que sincroniza los cambios y reinicia el servicio en una sola orden. Primero copia al servidor solo los archivos que han cambiado, excluyendo lo que no debe viajar (el repositorio Git, el entorno virtual, las credenciales locales y los PDF generados); después corrige el propietario de los archivos en el servidor; y por último reinicia el servicio para que cargue la versión nueva. Todo el ciclo de sincronizar y reiniciar lleva unos 15 segundos.

10.12 Decisiones de diseño técnico

Decisión	Por qué
Un solo archivo <code>app.py</code> en lugar de varios módulos	Con 22 rutas y 759 líneas, partir el archivo añade complejidad sin ninguna ventaja real.
Plantillas y rutas en castellano	Para que todo el proyecto sea coherente con el dominio (un taller español).
SQL directo	Con 5 tablas y consultas conocidas es más legible y se aprende mejor el lenguaje SQL.
PDF guardado en disco en vez de solo en memoria	Facilita depurar y permite reimprimir el mismo archivo después.
Quitar el modo de datos de prueba	El código duplicado en cada ruta se desincronizaba con la versión real, mejor un único camino contra MariaDB.

10.13 Capturas de la interfaz

A continuación se muestran las pantallas principales de RepairHub, que muestran el aspecto y el funcionamiento de la aplicación web.

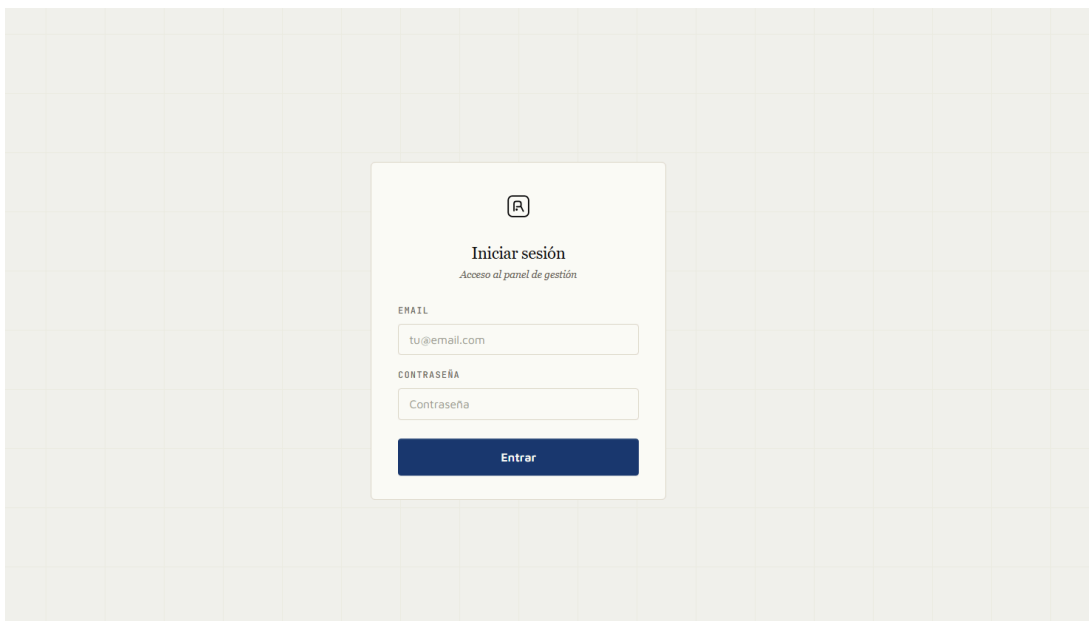


Figura 1. Pantalla de inicio de sesión.

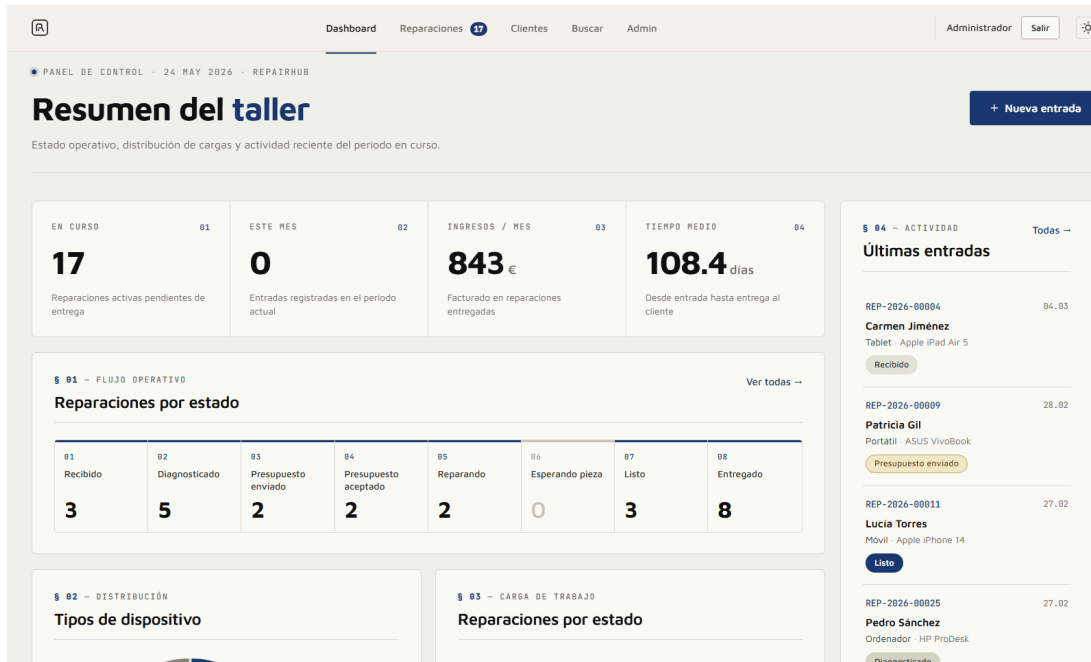


Figura 2. Panel principal (dashboard) tras iniciar sesión, con el resumen de la actividad del taller y los accesos rápidos.

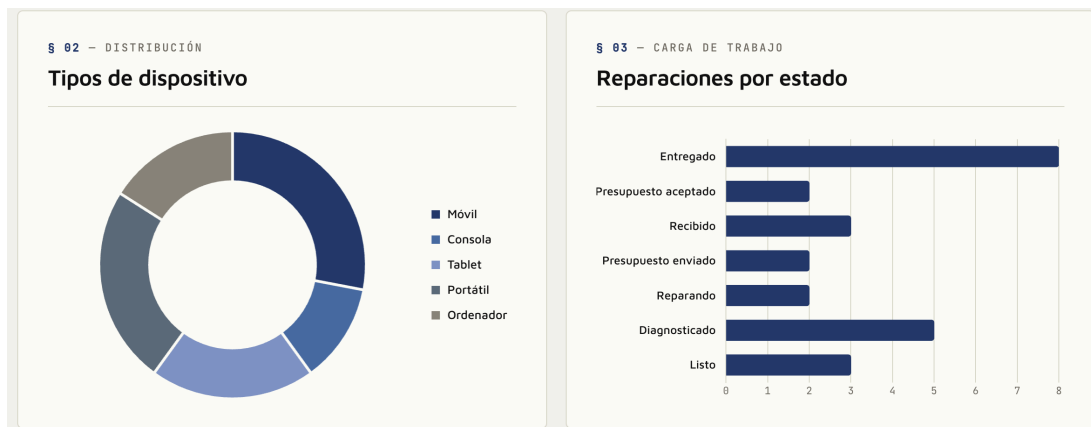


Figura 3. Vista de estadísticas del panel principal, con las métricas de reparaciones agrupadas por estado.

Dashboard Reparaciones 17 Cientes Buscar Admin Administrador Salir																																																						
Cientes (12)																																																						
<table> <tr> <th>NOMBRE</th><th>TELÉFONO</th><th>EMAIL</th><th>REPARACIONES</th><th>ALTA</th></tr> <tr> <td>Ana Martínez</td><td>—</td><td>ana@email.com</td><td>1</td><td>15/05/2026</td></tr> <tr> <td>Carlos García</td><td>612345678</td><td>carlos@email.com</td><td>4</td><td>15/05/2026</td></tr> <tr> <td>Carmen Jiménez</td><td>689012345</td><td>carmen@email.com</td><td>4</td><td>15/05/2026</td></tr> <tr> <td>Elena Ruiz</td><td>667890123</td><td>elena@email.com</td><td>1</td><td>15/05/2026</td></tr> <tr> <td>Javier Moreno</td><td>678901234</td><td>—</td><td>1</td><td>15/05/2026</td></tr> <tr> <td>Lucía Torres</td><td>601234567</td><td>—</td><td>2</td><td>15/05/2026</td></tr> <tr> <td>Luis Fernández</td><td>656789012</td><td>luis@email.com</td><td>1</td><td>15/05/2026</td></tr> <tr> <td>María López</td><td>623456789</td><td>maria@email.com</td><td>2</td><td>15/05/2026</td></tr> <tr> <td>Miguel Navarro</td><td>611223344</td><td>miguel@email.com</td><td>3</td><td>15/05/2026</td></tr> </table>					NOMBRE	TELÉFONO	EMAIL	REPARACIONES	ALTA	Ana Martínez	—	ana@email.com	1	15/05/2026	Carlos García	612345678	carlos@email.com	4	15/05/2026	Carmen Jiménez	689012345	carmen@email.com	4	15/05/2026	Elena Ruiz	667890123	elena@email.com	1	15/05/2026	Javier Moreno	678901234	—	1	15/05/2026	Lucía Torres	601234567	—	2	15/05/2026	Luis Fernández	656789012	luis@email.com	1	15/05/2026	María López	623456789	maria@email.com	2	15/05/2026	Miguel Navarro	611223344	miguel@email.com	3	15/05/2026
NOMBRE	TELÉFONO	EMAIL	REPARACIONES	ALTA																																																		
Ana Martínez	—	ana@email.com	1	15/05/2026																																																		
Carlos García	612345678	carlos@email.com	4	15/05/2026																																																		
Carmen Jiménez	689012345	carmen@email.com	4	15/05/2026																																																		
Elena Ruiz	667890123	elena@email.com	1	15/05/2026																																																		
Javier Moreno	678901234	—	1	15/05/2026																																																		
Lucía Torres	601234567	—	2	15/05/2026																																																		
Luis Fernández	656789012	luis@email.com	1	15/05/2026																																																		
María López	623456789	maria@email.com	2	15/05/2026																																																		
Miguel Navarro	611223344	miguel@email.com	3	15/05/2026																																																		

Figura 4. Listado de clientes registrados, con buscador y acceso a la ficha individual de cada cliente.

Dashboard Reparaciones 17 Cientes Buscar Admin Administrador Salir					
Reparaciones (25) Exportar CSV					
Todos Recibido Diagnosticado Presupuesto enviado Presupuesto aceptado Reparando Esperando pieza Listo Entregado					
DESDE dd/mm/aaaa HASTA dd/mm/aaaa TIPO DISPOSITIVO Todos CLIENTE Nombre del cliente Filtrar Limpiar					
CÓDIGO	CLIENTE	DISPOSITIVO	AVERÍA	ESTADO	FECHA
REP-2026-00004	Carmen Jiménez	Tablet Apple	Touch no responde	Recibido	04/03/2026
REP-2026-00009	Patricia Gil	Portátil ASUS	Ventilador ruidoso	Presupuesto enviado	28/02/2026
REP-2026-00011	Lucía Torres	Móvil Apple	No enciende	Listo	27/02/2026
REP-2026-00025	Pedro Sánchez	Ordenador HP	Fuente de alimentación	Diagnosticado	27/02/2026
REP-2026-00015	Javier Moreno	Portátil Lenovo	Muy lento	Listo	19/02/2026
REP-2026-00023	Carlos García	Móvil Apple	Pantalla rota	Entregado	17/02/2026
REP-2026-00001	Ana Martínez	Móvil OnePlus	No enciende	Entregado	13/02/2026

Figura 5. Listado de reparaciones, donde se muestra el estado actual y los datos principales de cada una.

R

Dashboard
Reparaciones 17
Clientes
Buscar
Admin

Administrador
Salir

Panel de administración

CREAR NUEVO USUARIO

NOMBRE

Nombre del técnico

EMAIL

email@repairhub.com

CONTRASEÑA

Contraseña

ROL

Técnico

Crear usuario

USUARIOS REGISTRADOS

NOMBRE	EMAIL	ROL	
Administrador	admin@repairhub.com	Admin	Tu
Roberto	roberto@repairhub.com	Técnico	Eliminar

Figura 6. Panel de administración, reservado al rol administrador, para la gestión de usuarios del sistema.

REP-2026-00004

Recibido

CLIENTE

Carmen Jiménez

Tel: 689012345

Email: carmen@email.com

DISPOSITIVO

Tablet — Apple iPad Air 5

Avería: Touch no responde

PRESUPUESTO / PRECIO

Presupuesto: — €

Precio final: — €

CAMBIAZ ESTADO

Seleccionar estado...

Cambiar estado

Reimprimir PDF

Editar datos

Eliminar

Historial

ESTADO	TÉCNICO	FECHA
Recibido	Roberto	04/03/2026 05:00

Figura 7. Ejemplo de informe en PDF generado automáticamente por la aplicación para una reparación.

34

11. ESTUDIO DE RESULTADOS

En este apartado se compara lo que se esperaba conseguir (apartado 6) con lo que realmente se ha conseguido, y se cuentan los problemas que salieron y cómo se resolvieron.

11.1 Plan de pruebas

Las pruebas son funcionales y manuales, hechas sobre la aplicación ya desplegada en producción. Para cada caso se parte de un estado conocido a partir de los datos de prueba, se ejecuta la operación y se comprueba el resultado y cuando hace falta también en la base de datos o en los archivos generados (PDF, CSV).

11.2 Resultados por bloque funcional

Tabla 13. *Plan de pruebas y resultados.*

Bloque	Caso probado	Resultado
Autenticación	Acceso a rutas protegidas sin sesión	Redirige al inicio de sesión
Autenticación	Acceso a la administración con rol técnico	Bloqueado
Autenticación	Login con credenciales correctas e incorrectas	Aceptado / rechazado con mensaje genérico
Reparaciones	Alta con campos válidos	Código generado y PDF disponible
Reparaciones	Alta sin teléfono ni correo	Rechazada con mensaje de error
Reparaciones	Alta con cliente ya existente	Cliente reutilizado, no se duplica
Reparaciones	Cambios de estado legales e ilegales	Aceptados o rechazados según el flujo
Reparaciones	Asignar presupuesto y precio final	Se guardan correctamente
Reparaciones	Edición y eliminación	Funcionan
PDF	Generación y reimpresión	PDF legible con resguardo y etiqueta
Búsqueda	Búsqueda por código y por nombre	Resultados correctos
Búsqueda	Autocompletado al rellenar el teléfono	Campos rellenados solos
Panel	Indicadores frente a consulta manual	Coinciden
Panel	Gráficos en tema claro y oscuro	Correctos en ambos
Listados	Paginación, filtros y exportación CSV	Funcionan según lo previsto
Interfaz	Cambio de tema y persistencia	Tema mantenido tras recargar
Interfaz	Pantalla de 360 px	Operativa
Despliegue	Servicio activo tras reiniciar el servidor	Sí
Despliegue	Ejecución del script de despliegue	Cambios aplicados sin error

11.3 Resultados frente a los indicadores definidos

Los indicadores de la Tabla 4 se comparan con lo conseguido de verdad:

Indicador	Umbral	Resultado	Cumplido
Número de rutas HTTP funcionales	≥ 20	22	Sí
Número de tablas relacionadas	5	5	Sí
Disponibilidad tras reinicio	Sí	Sí	Sí
Rutas protegidas sin sesión	Todas	Todas	Sí
Contraseñas en <i>hash</i>	Sí	Sí	Sí
PDF imprimible desde el navegador	Sí	Sí	Sí
Operación en pantalla de 360 px	Sí	Sí	Sí
Latencia de la búsqueda	Inmediata	Inmediata en red local	Sí
Despliegue automatizado en un paso	Sí	Sí	Sí
Memoria con 13 apartados + anexo	Sí	Sí	Sí

11.4 Incidencias detectadas y resolución

Incidencia (fase)	Resolución
El PDF se abría dos veces al crear la nueva entrada (F6)	Se unificó el flujo en una sola pestaña con una ventana emergente.
Variables CSS que no existían usadas en la ventana emergente (F6)	Se añadieron las que faltaban.
Clases CSS sin reglas asociadas (F6)	Se definieron los estilos correspondientes.
Credenciales escritas a mano en el script de datos de prueba (F8)	Se cambió para que lea la configuración común.
HestiaCP no soportado en Debian 13 (F8)	Se instaló el <i>stack</i> a mano.
Código muerto del modo de datos de prueba (F8)	Se limpió por completo el archivo principal.

La incidencia más relevante fue la de HestiaCP (apartado 10.11.2): obligó a cambiar el plan a mitad de despliegue, pero se resolvió manteniendo Debian 13 e instalando todo manualmente, lo que aportó más aprendizaje que el plan original.

11.5 Cumplimiento de objetivos

Los nueve objetivos específicos del apartado 3.3.2 se comparan con lo conseguido:

Objetivo	Estado	Evidencia
OE1 — Modelo de datos relacional	Cumplido	Cinco tablas con claves foráneas.
OE2 — Backend con ≥ 20 rutas	Cumplido	22 rutas funcionales.
OE3 — Interfaz minimalista y responsive	Cumplido	CSS único, tema dual claro/oscuro, dos puntos de ruptura.
OE4 — PDF A4 con resguardo y etiqueta	Cumplido	Módulo generador de PDF y ruta para servirlo.
OE5 — Autenticación con dos roles	Cumplido	Decoradores, campo de rol, contraseñas en <i>hash</i> .
OE6 — Despliegue en MV Debian	Cumplido	Servidor 192.168.1.143 con Unicorn, Nginx y systemd.
OE7 — Script de despliegue	Cumplido	Sincronización y reinicio en unos 15 s.
OE8 — Plan de pruebas funcionales	Cumplido	Apartado 11.2.
OE9 — Documentación completa	Cumplido	Esta memoria.

12. CONCLUSIONES

12.1 Síntesis de logros

Se ha conseguido el objetivo general: una web funcional que cubre el ciclo completo de gestión de un taller, en marcha en producción sobre Debian 13 con MariaDB real, con 22 rutas HTTP, dos roles de usuario, generación de PDF y exportación a CSV. Los nueve objetivos específicos están cumplidos y todos los indicadores de éxito se han alcanzado o superado.

12.2 Aprendizajes profesionales

El proyecto me ha servido para aprender varias cosas útiles. El ciclo de despliegue completo: crear una máquina virtual, instalar Linux mínimo, configurar la base de datos, levantar Nginx como *proxy* inverso, integrar la aplicación con *systemd* y automatizar el despliegue con un *script*. Aceptar imprevistos y adaptar el plan: que HestiaCP no funcionara en Debian 13 obligó a montar el servidor a mano, y al final fue lo que más se aprendió. Y que una primera versión que funcione importa más que un diseño perfecto a medias porque las mejoras visuales llegaron después de comprobar que todo funcionaba correctamente.

12.3 Limitaciones del trabajo

No soporta HTTPS, así que su uso queda limitado a la red local. No hace copias de seguridad automáticas de la base de datos. El idioma está fijado en castellano, no hay mas idiomas.

12.4 Líneas futuras de mejora

Avisos automáticos al cliente por SMS o correo, inventario de piezas con alerta de *stock*, facturación con IVA y galería de fotos por reparación. Encuesta de satisfacción al cliente, comparativa de estadísticas entre años e importación masiva desde Excel.

12.5 Reflexión final

Este proyecto es la demostración de que se es capaz de pensar, planificar, construir, desplegar y documentar una solución para una necesidad real, juntando lo aprendido en el ciclo SMR y aprendizaje individual. La web/herramienta final está lo bastante terminada como para que un taller pequeño pudiera empezar a usarla, con la ventaja de ser *software* libre, autoalojado y entendible pieza a pieza.

Como punto extra me gustaría destacar que esta herramienta voy a mejorarla y publicarla por mi cuenta como herramienta de software libre/código abierto, crearé una web estática con SEO y una guía, por si algún negocio un día necesita algo similar, que con una búsqueda pueda encontrarla y usarla sin coste. Mediante docker y pocos comandos podrá ejecutarla, personalizarla a su gusto y demás. Esta información no es sobre el proyecto pero me parecía buena idea comentarlo. Pienso que esta herramienta es muy útil para estos negocios, no me cuesta nada mejorarla y ofrecerla gratuitamente solo por ayudar.

Enlace al repositorio de la herramienta: <https://github.com/robertocarrascoso/OpenRepair>

13. REFERENCIAS BIBLIOGRÁFICAS

Documentación oficial consultada durante el desarrollo:

- Flask. <https://flask.palletsprojects.com/>
- Jinja2. <https://jinja.palletsprojects.com/>
- Werkzeug (utilidades de seguridad y *hash* de contraseñas).
<https://werkzeug.palletsprojects.com/>
- python-dotenv. <https://pypi.org/project/python-dotenv/>
- MariaDB. <https://mariadb.org/documentation/>
- MySQL Connector/Python. <https://dev.mysql.com/doc/connector-python/en/>
- ReportLab (generación de PDF). <https://docs.reportlab.com/>
- Gunicorn. <https://docs.gunicorn.org/>
- Nginx. <https://nginx.org/en/docs/>
- Chart.js. <https://www.chartjs.org/docs/latest/>
- HestiaCP. <https://hestiacp.com/docs/>
- VirtualBox. <https://www.virtualbox.org/manual/>
- Tipografía Inter (SIL Open Font License). <https://rsms.me/inter/>

ANEXO A. CÓDIGO FUENTE COMPLETO DEL PROYECTO

Código fuente de la aplicación RepairHub desarrollada para este Proyecto Intermodular. Los archivos se presentan agrupados por capas siguiendo la arquitectura descrita en el apartado 10: configuración, base de datos, aplicación Flask, utilidades, plantillas Jinja2 y recursos estáticos. El repositorio público del proyecto está disponible en https://github.com/robertocarrascoso/TFGM_RepairHub.

A.1. Configuración

Archivos de configuración de la aplicación Flask y lista de dependencias del proyecto.

A.1.1. config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

DB_CONFIG = {
    'host': os.getenv('DB_HOST', 'localhost'),
    'user': os.getenv('DB_USER', 'repairhub_user'),
    'password': os.getenv('DB_PASSWORD', ''),
    'database': os.getenv('DB_NAME', 'repairhub')
}

SECRET_KEY = os.getenv('SECRET_KEY', 'clave-de-desarrollo')
```

A.1.2. requirements.txt

```
flask
mysql-connector-python
python-dotenv
reportlab
```

A.2. Base de datos

Esquema MariaDB del sistema y script de carga inicial de datos de prueba.

A.2.1. base-de-datos/schema.sql

```
CREATE DATABASE IF NOT EXISTS repairhub
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;

USE repairhub;

-- Tabla de usuarios (login)
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(256) NOT NULL,
```

```

    rol ENUM('admin', 'tecnico') NOT NULL DEFAULT 'tecnico',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Tabla de los clientes
CREATE TABLE clientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    telefono VARCHAR(20),
    email VARCHAR(100),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Tabla de las reparaciones
CREATE TABLE reparaciones (
    id INT AUTO_INCREMENT PRIMARY KEY,
    codigo VARCHAR(20) NOT NULL UNIQUE,
    cliente_id INT NOT NULL,
    tipo_dispositivo VARCHAR(50) NOT NULL,
    marca VARCHAR(50),
    modelo VARCHAR(50),
    averia TEXT NOT NULL,
    observaciones TEXT,
    estado VARCHAR(30) DEFAULT 'Recibido',
    presupuesto DECIMAL(8,2),
    precio_final DECIMAL(8,2),
    presupuesto_aceptado BOOLEAN DEFAULT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);

-- Contador de reparaciones por año (nunca retrocede)
CREATE TABLE contadores (
    year INT PRIMARY KEY,
    ultimo_num INT NOT NULL DEFAULT 0
);

-- Tabla del historial de estados
CREATE TABLE historial_estados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    reparacion_id INT NOT NULL,
    estado VARCHAR(30) NOT NULL,
    tecnico VARCHAR(50),
    fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (reparacion_id) REFERENCES reparaciones(id)
);

```

A.2.2. base-de-datos/seed.py

```

"""seed.py – Insertar datos de prueba en la base de datos."""
import sys
import os
import mysql.connector
import random
from datetime import datetime, timedelta
from werkzeug.security import generate_password_hash

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from config import DB_CONFIG

conn = mysql.connector.connect(**DB_CONFIG)
cursor = conn.cursor()

TABLAS = ('historial_estados', 'reparaciones', 'clientes', 'usuarios')
for tabla in TABLAS:

```

```

        cursor.execute(f"DELETE FROM {tabla}")
    for tabla in TABLAS:
        cursor.execute(f"ALTER TABLE {tabla} AUTO_INCREMENT = 1")

    usuarios = [
        ('Administrador', 'admin@repairhub.com', 'admin123', 'admin'),
        ('Roberto', 'roberto@repairhub.com', 'tecnico123', 'tecnico'),
    ]
    for nombre, email, password, rol in usuarios:
        cursor.execute(
            "INSERT INTO usuarios (nombre, email, password_hash, rol) VALUES (%s, %s, %s, %s)",
            (nombre, email, generate_password_hash(password), rol)
        )
    conn.commit()

    clientes = [
        ('Carlos García', '612345678', 'carlos@email.com'),
        ('María López', '623456789', 'maria@email.com'),
        ('Pedro Sánchez', '634567890', ''),
        ('Ana Martínez', '', 'ana@email.com'),
        ('Luis Fernández', '656789012', 'luis@email.com'),
        ('Elena Ruiz', '667890123', 'elena@email.com'),
        ('Javier Moreno', '678901234', ''),
        ('Carmen Jiménez', '689012345', 'carmen@email.com'),
        ('Roberto Díaz', '690123456', 'roberto@email.com'),
        ('Lucía Torres', '601234567', ''),
        ('Miguel Navarro', '611223344', 'miguel@email.com'),
        ('Patricia Gil', '622334455', 'patricia@email.com'),
    ]
    for nombre, tel, email in clientes:
        cursor.execute("INSERT INTO clientes (nombre, telefono, email) VALUES (%s, %s, %s)", (nombre, tel, email))
    conn.commit()

    tipos = ['Móvil', 'Tablet', 'Portátil', 'Ordenador', 'Consola']
    marcas_modelos = {
        'Móvil': [('Samsung', 'Galaxy S23'), ('Apple', 'iPhone 14'), ('Xiaomi', 'Redmi Note 13'), ('OnePlus', '12')],
        'Tablet': [('Apple', 'iPad Air 5'), ('Samsung', 'Galaxy Tab S9'), ('Lenovo', 'Tab P11')],
        'Portátil': [('HP', 'Pavilion 15'), ('Lenovo', 'ThinkPad T14'), ('Dell', 'XPS 13'), ('ASUS', 'VivoBook')],
        'Ordenador': [('Custom', 'Sobremesa'), ('HP', 'ProDesk'), ('Lenovo', 'ThinkCentre')],
        'Consola': [('Sony', 'PS5'), ('Microsoft', 'Xbox Series X'), ('Nintendo', 'Switch')],
    }
    averias = {
        'Móvil': ['Pantalla rota', 'No carga', 'Batería hinchada', 'Cámara borrosa', 'No enciende'],
        'Tablet': ['Pantalla rota', 'Puerto de carga dañado', 'Batería dura poco', 'Touch no responde'],
        'Portátil': ['No enciende', 'Pantalla en negro', 'Teclado no funciona', 'Muy lento', 'Ventilador ruidoso'],
        'Ordenador': ['Pantalla azul', 'No arranca', 'Disco duro roto', 'Fuente de alimentación'],
        'Consola': ['No lee discos', 'Se sobrecalienta', 'Mando no conecta', 'Error de sistema'],
    }
    estados_completo = ['Recibido', 'Diagnosticado', 'Presupuesto enviado', 'Presupuesto aceptado', 'Reparando', 'Listo', 'Entregado']

    for i in range(1, 26):
        tipo = random.choice(tipos)
        marca, modelo = random.choice(marcas_modelos[tipo])
        averia = random.choice(averias[tipo])
        cliente_id = random.randint(1, len(clientes))
        fecha_base = datetime(2026, 1, 1) + timedelta(days=random.randint(0, 65))

        num_estados = random.randint(1, len(estados_completo))
        estado_actual = estados_completo[num_estados - 1]

        presupuesto = round(random.uniform(30, 200), 2) if num_estados >= 3 else None
        # Rechazo de presupuesto: 20% de los que están en "Presupuesto enviado"
        aceptado = True if num_estados >= 4 else (False if num_estados == 3 and random.random() < 0.2 else None)
        precio = presupuesto if num_estados >= 6 else None

        codigo = f"REP-2026-{i:05d}"

```

```

        cursor.execute("""
            INSERT INTO reparaciones (codigo, cliente_id, tipo_dispositivo, marca, modelo, averia, estado,
presupuesto, precio_final, presupuesto_aceptado, created_at)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            """, (codigo, cliente_id, tipo, marca, modelo, averia, estado_actual, presupuesto, precio, aceptado,
fecha_base))

        rep_id = cursor.lastrowid

        for j in range(num_estados):
            fecha_estado = fecha_base + timedelta(days=j, hours=random.randint(1, 8))
            cursor.execute("INSERT INTO historial_estados (reparacion_id, estado, tecnico, fecha) VALUES (%s, %s,
%s, %s)",
                            (rep_id, estados_completo[j], 'Roberto', fecha_estado))

conn.commit()
cursor.close()
conn.close()
print(f"Seed completado: {len(usuarios)} usuarios, {len(clientes)} clientes, 25 reparaciones.")

```

A.3. Aplicacion Flask

Punto de entrada de la aplicación con todas las rutas HTTP, controladores y lógica de negocio.

A.3.1. app.py

```

from flask import Flask, render_template, request, redirect, url_for, flash, jsonify, send_file, session,
Response
from werkzeug.security import generate_password_hash, check_password_hash
from functools import wraps
from config import DB_CONFIG, SECRET_KEY
from utilidades.pdf_generator import generar_pdf
from datetime import datetime, date
import csv
import io
import os

app = Flask(__name__,
            template_folder='plantillas',
            static_folder='estaticos')
app.secret_key = SECRET_KEY

def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if 'user_id' not in session:
            flash('Inicia sesión para continuar.', 'error')
            return redirect(url_for('login'))
        return f(*args, **kwargs)
    return decorated

def admin_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if 'user_id' not in session:
            flash('Inicia sesión para continuar.', 'error')
            return redirect(url_for('login'))
        if session.get('user_rol') != 'admin':
            flash('Acceso restringido a administradores.', 'error')
            return redirect(url_for('dashboard'))
    return decorated

```

```

        return f(*args, **kwargs)
    return decorated

def get_db():
    import mysql.connector
    return mysql.connector.connect(**DB_CONFIG)

def _construir_where_filtros(filtro, fecha_desde, fecha_hasta, tipo_dispositivo, cliente_nombre):
    where, params = [], []
    if filtro != 'todos':
        where.append("r.estado = %s"); params.append(filtro)
    if fecha_desde:
        where.append("DATE(r.created_at) >= %s"); params.append(fecha_desde)
    if fecha_hasta:
        where.append("DATE(r.created_at) <= %s"); params.append(fecha_hasta)
    if tipo_dispositivo:
        where.append("r.tipo_dispositivo = %s"); params.append(tipo_dispositivo)
    if cliente_nombre:
        where.append("c.nombre LIKE %s"); params.append(f'%{cliente_nombre}%')
    where_sql = (" WHERE " + " AND ".join(where)) if where else ""
    return where_sql, params

@app.context_processor
def inject_pendientes():
    if 'user_id' not in session:
        return {'pendientes_count': 0}
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT COUNT(*) as total FROM reparaciones WHERE estado != 'Entregado'")
    count = cursor.fetchone()['total']
    cursor.close()
    db.close()
    return {'pendientes_count': count}

@app.route('/login', methods=['GET', 'POST'])
def login():
    if 'user_id' in session:
        return redirect(url_for('dashboard'))

    if request.method == 'POST':
        email = request.form.get('email', '').strip()
        password = request.form.get('password', '')

        db = get_db()
        cursor = db.cursor(dictionary=True)
        cursor.execute("SELECT * FROM usuarios WHERE email = %s", (email,))
        user = cursor.fetchone()
        cursor.close()
        db.close()

        if user and check_password_hash(user['password_hash'], password):
            session['user_id'] = user['id']
            session['user_nombre'] = user['nombre']
            session['user_rol'] = user['rol']
            return redirect(url_for('dashboard'))

        flash('Email o contraseña incorrectos.', 'error')
        return redirect(url_for('login'))

    return render_template('login.html')

```

```

@app.route('/logout')
def logout():
    session.clear()
    flash('Sesión cerrada.', 'success')
    return redirect(url_for('login'))

@app.route('/')
@login_required
def dashboard():
    db = get_db()
    cursor = db.cursor(dictionary=True)

    cursor.execute("SELECT COUNT(*) as total FROM reparaciones WHERE estado != 'Entregado'")
    pendientes = cursor.fetchone()['total']

    cursor.execute("SELECT COUNT(*) as total FROM reparaciones WHERE MONTH(created_at) = MONTH(NOW()) AND YEAR(created_at) = YEAR(NOW())")
    este_mes = cursor.fetchone()['total']

    cursor.execute("SELECT COALESCE(SUM(precio_final), 0) as total FROM reparaciones WHERE estado = 'Entregado' AND MONTH(updated_at) = MONTH(NOW()) AND YEAR(updated_at) = YEAR(NOW())")
    ingresos_mes = cursor.fetchone()['total']

    cursor.execute("""
        SELECT AVG(DATEDIFF(updated_at, created_at)) as media
        FROM reparaciones WHERE estado = 'Entregado'
    """)
    row_media = cursor.fetchone()
    tiempo_medio = round(row_media['media'], 1) if row_media['media'] else 0

    cursor.execute("""
        SELECT r.*, c.nombre as cliente_nombre
        FROM reparaciones r
        JOIN clientes c ON r.cliente_id = c.id
        ORDER BY r.created_at DESC LIMIT 5
    """)
    ultimas = cursor.fetchall()

    cursor.execute("SELECT estado, tipo_dispositivo FROM reparaciones")
    reparaciones = cursor.fetchall()

    cursor.close()
    db.close()

    return render_template('dashboard.html',
        pendientes=pendientes,
        este_mes=este_mes,
        ingresos_mes=ingresos_mes,
        tiempo_medio=tiempo_medio,
        ultimas=ultimas,
        reparaciones=reparaciones
    )

@app.route('/nueva-entrada', methods=['GET', 'POST'])
@login_required
def nueva_entrada():
    if request.method == 'POST':
        nombre = request.form.get('nombre', '').strip()
        telefono = request.form.get('telefono', '').strip()
        email = request.form.get('email', '').strip()
        tipo = request.form.get('tipo_dispositivo', '').strip()
        marca = request.form.get('marca', '').strip()

```

```

modelo = request.form.get('modelo', '').strip()
averia = request.form.get('averia', '').strip()
observaciones = request.form.get('observaciones', '').strip()

if not nombre or not averia or not tipo:
    flash('Nombre, avería y tipo de dispositivo son obligatorios.', 'error')
    return redirect(url_for('nueva_entrada'))

if not telefono and not email:
    flash('Indica al menos un dato de contacto (teléfono o email).', 'error')
    return redirect(url_for('nueva_entrada'))

db = get_db()
cursor = db.cursor(dictionary=True)

cursor.execute("SELECT * FROM clientes WHERE telefono = %s OR email = %s", (telefono, email))
cliente = cursor.fetchone()

if not cliente:
    cursor.execute("INSERT INTO clientes (nombre, telefono, email) VALUES (%s, %s, %s)", (nombre,
telefono, email))
    db.commit()
    cliente_id = cursor.lastrowid
else:
    cliente_id = cliente['id']

# Contador atómico por año para evitar códigos duplicados en concurrencia
year = datetime.now().year
cursor.execute("INSERT INTO contadores (year, ultimo_num) VALUES (%s, 1) ON DUPLICATE KEY UPDATE
ultimo_num = ultimo_num + 1", (year,))
cursor.execute("SELECT ultimo_num FROM contadores WHERE year = %s", (year,))
num = cursor.fetchone()['ultimo_num']
codigo = f"REP-{year}-{num:05d}"

cursor.execute("""
INSERT INTO reparaciones (codigo, cliente_id, tipo_dispositivo, marca, modelo, averia,
observaciones)
VALUES (%s, %s, %s, %s, %s, %s, %s)
""", (codigo, cliente_id, tipo, marca, modelo, averia, observaciones))
reparacion_id = cursor.lastrowid

cursor.execute("INSERT INTO historial_estados (reparacion_id, estado, tecnico) VALUES (%s, 'Recibido',
%s)", (reparacion_id, session.get('user_nombre', 'Técnico')))
db.commit()
cursor.close()
db.close()

return redirect(url_for('nueva_entrada', pdf=codigo))

return render_template('nueva_entrada.html')

PER_PAGE = 20

def _get_filtros_from_request():
    filtro = request.args.get('estado', 'todos')
    fecha_desde_str = request.args.get('fecha_desde', '')
    fecha_hasta_str = request.args.get('fecha_hasta', '')
    tipo_dispositivo = request.args.get('tipo_dispositivo', '')
    cliente_nombre = request.args.get('cliente', '')
    page = request.args.get('page', 1, type=int)
    if page < 1:
        page = 1

```



```

    fecha_desde = None
    fecha_hasta = None
    if fecha_desde_str:
        try:
            fecha_desde = date.fromisoformat(fecha_desde_str)
        except ValueError:
            pass
    if fecha_hasta_str:
        try:
            fecha_hasta = date.fromisoformat(fecha_hasta_str)
        except ValueError:
            pass

    return filtro, fecha_desde, fecha_hasta, tipo_dispositivo, cliente_nombre, page, fecha_desde_str,
    fecha_hasta_str

@app.route('/reparaciones')
@login_required
def reparaciones():
    filtro, fecha_desde, fecha_hasta, tipo_dispositivo, cliente_nombre, page, fecha_desde_str, fecha_hasta_str =
    _get_filtros_from_request()

    db = get_db()
    cursor = db.cursor(dictionary=True)

    where_sql, params = _construir_where_filtros(filtro, fecha_desde, fecha_hasta, tipo_dispositivo,
    cliente_nombre)

    cursor.execute(f"SELECT COUNT(*) as total FROM reparaciones r JOIN clientes c ON r.cliente_id =
    c.id{where_sql}", params)
    total = cursor.fetchone()['total']
    total_pages = max(1, (total + PER_PAGE - 1) // PER_PAGE)
    page = min(page, total_pages)
    offset = (page - 1) * PER_PAGE

    cursor.execute(f"""
        SELECT r.*, c.nombre as cliente_nombre
        FROM reparaciones r JOIN clientes c ON r.cliente_id = c.id
        {where_sql}
        ORDER BY r.created_at DESC
        LIMIT %s OFFSET %s
    """, params + [PER_PAGE, offset])
    lista = cursor.fetchall()

    cursor.execute("SELECT DISTINCT tipo_dispositivo FROM reparaciones ORDER BY tipo_dispositivo")
    tipos = [row['tipo_dispositivo'] for row in cursor.fetchall()]

    cursor.close()
    db.close()
    return render_template('reparaciones.html',
        reparaciones=lista, filtro_actual=filtro,
        page=page, total_pages=total_pages, total=total,
        fecha_desde=fecha_desde_str, fecha_hasta=fecha_hasta_str,
        tipo_dispositivo=tipo_dispositivo, cliente=cliente_nombre,
        tipos_dispositivo=tipos)

FLUJO_ESTADOS = {
    'Recibido': ['Diagnosticado'],
    'Diagnosticado': ['Presupuesto enviado'],
    'Presupuesto enviado': ['Presupuesto aceptado', 'Entregado'], # "Entregado" aquí = presupuesto rechazado
    'Presupuesto aceptado': ['Reparando'],
    'Reparando': ['Esperando pieza', 'Listo'],
    'Esperando pieza': ['Reparando'],

```

```

        'Listo': ['Entregado'],
        'Entregado': []
    }

@app.route('/reparacion/<codigo>')
@login_required
def detalle_reparacion(codigo):
    db = get_db()
    cursor = db.cursor(dictionary=True)

    cursor.execute("""
        SELECT r.*, c.nombre as cliente_nombre, c.telefono as cliente_telefono,
               c.email as cliente_email, c.id as cid
        FROM reparaciones r JOIN clientes c ON r.cliente_id = c.id
        WHERE r.codigo = %s
    """, (codigo,))
    rep = cursor.fetchone()

    if not rep:
        cursor.close()
        db.close()
        flash('Reparación no encontrada.', 'error')
        return redirect(url_for('reparaciones'))

    cursor.execute("SELECT * FROM historial_estados WHERE reparacion_id = %s ORDER BY fecha DESC", (rep['id'],))
    historial = cursor.fetchall()

    cursor.close()
    db.close()

    estados_posibles = FLUJO_ESTADOS.get(rep['estado'], [])
    return render_template('reparacion.html', rep=rep, historial=historial, estados_posibles=estados_posibles)

@app.route('/reparacion/<codigo>/cambiar-estado', methods=['POST'])
@login_required
def cambiar_estado(codigo):
    nuevo_estado = request.form.get('nuevo_estado')

    db = get_db()
    cursor = db.cursor(dictionary=True)

    cursor.execute("SELECT * FROM reparaciones WHERE codigo = %s", (codigo,))
    rep = cursor.fetchone()

    if rep and nuevo_estado in FLUJO_ESTADOS.get(rep['estado'], []):
        cursor.execute("UPDATE reparaciones SET estado = %s WHERE codigo = %s", (nuevo_estado, codigo))

        if nuevo_estado == 'Presupuesto aceptado':
            cursor.execute("UPDATE reparaciones SET presupuesto_aceptado = TRUE WHERE codigo = %s", (codigo,))
        elif nuevo_estado == 'Entregado' and rep.get('presupuesto') and not rep.get('presupuesto_aceptado'):
            cursor.execute("UPDATE reparaciones SET presupuesto_aceptado = FALSE WHERE codigo = %s", (codigo,))

        cursor.execute("INSERT INTO historial_estados (reparacion_id, estado, tecnico) VALUES (%s, %s, %s)",
            (rep['id'], nuevo_estado, session.get('user_nombre', 'Técnico')))
        db.commit()
        flash(f'Estado cambiado a "{nuevo_estado}".', 'success')
    else:
        flash('Cambio de estado no válido.', 'error')

    cursor.close()
    db.close()
    return redirect(url_for('detalle_reparacion', codigo=codigo))

```

```

@app.route('/reparacion/<codigo>/presupuesto', methods=['POST'])
@login_required
def enviar_presupuesto(codigo):
    presupuesto = request.form.get('presupuesto', type=float)

    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT * FROM reparaciones WHERE codigo = %s", (codigo,))
    rep = cursor.fetchone()

    if rep and rep['estado'] == 'Diagnosticado':
        cursor.execute("UPDATE reparaciones SET presupuesto = %s, estado = 'Presupuesto enviado' WHERE codigo = %s", (presupuesto, codigo))
        cursor.execute("INSERT INTO historial_estados (reparacion_id, estado, tecnico) VALUES (%s, 'Presupuesto enviado', %s)", (rep['id'], session.get('user_nombre', 'Técnico')))
        db.commit()
        flash(f'Presupuesto de {presupuesto}€ enviado.', 'success')

    cursor.close()
    db.close()
    return redirect(url_for('detalle_reparacion', codigo=codigo))

@app.route('/reparacion/<codigo>/precio-final', methods=['POST'])
@login_required
def precio_final(codigo):
    precio = request.form.get('precio_final', type=float)

    db = get_db()
    cursor = db.cursor()
    cursor.execute("UPDATE reparaciones SET precio_final = %s WHERE codigo = %s", (precio, codigo))
    db.commit()
    cursor.close()
    db.close()
    flash(f'Precio final: {precio}€.', 'success')
    return redirect(url_for('detalle_reparacion', codigo=codigo))

@app.route('/reparacion/<codigo>/editar', methods=['GET', 'POST'])
@login_required
def editar_reparacion(codigo):
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT * FROM reparaciones WHERE codigo = %s", (codigo,))
    rep = cursor.fetchone()

    if not rep:
        cursor.close()
        db.close()
        flash('Reparación no encontrada.', 'error')
        return redirect(url_for('reparaciones'))

    if request.method == 'POST':
        tipo = request.form.get('tipo_dispositivo', '').strip() or rep['tipo_dispositivo']
        marca = request.form.get('marca', '').strip()
        modelo = request.form.get('modelo', '').strip()
        averia = request.form.get('averia', '').strip() or rep['averia']
        observaciones = request.form.get('observaciones', '').strip()

        cursor.execute("""
            UPDATE reparaciones SET tipo_dispositivo=%s, marca=%s, modelo=%s,
            averia=%s, observaciones=%s WHERE codigo=%s
            """, (tipo, marca, modelo, averia, observaciones, codigo))
        db.commit()
        cursor.close()
        db.close()

```

```

        flash('Reparación actualizada.', 'success')
        return redirect(url_for('detalle_reparacion', codigo=codigo))

    cursor.close()
    db.close()
    return render_template('editar_reparacion.html', rep=rep)

@app.route('/reparacion/<codigo>/eliminar', methods=['POST'])
@login_required
def eliminar_reparacion(codigo):
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT id FROM reparaciones WHERE codigo = %s", (codigo,))
    rep = cursor.fetchone()

    if not rep:
        cursor.close()
        db.close()
        flash('Reparación no encontrada.', 'error')
        return redirect(url_for('reparaciones'))

    cursor.execute("DELETE FROM historial_estados WHERE reparacion_id = %s", (rep['id'],))
    cursor.execute("DELETE FROM reparaciones WHERE id = %s", (rep['id'],))
    db.commit()
    cursor.close()
    db.close()
    flash(f'Reparación {codigo} eliminada.', 'success')
    return redirect(url_for('reparaciones'))

@app.route('/clientes')
@login_required
def clientes():
    page = request.args.get('page', 1, type=int)
    if page < 1:
        page = 1

    db = get_db()
    cursor = db.cursor(dictionary=True)

    cursor.execute("SELECT COUNT(*) as total FROM clientes")
    total = cursor.fetchone()['total']
    total_pages = max(1, (total + PER_PAGE - 1) // PER_PAGE)
    page = min(page, total_pages)
    offset = (page - 1) * PER_PAGE

    cursor.execute("""
        SELECT c.*, COUNT(r.id) as n_reparaciones
        FROM clientes c
        LEFT JOIN reparaciones r ON c.id = r.cliente_id
        GROUP BY c.id
        ORDER BY c.nombre
        LIMIT %s OFFSET %s
    """, (PER_PAGE, offset))
    lista = cursor.fetchall()
    cursor.close()
    db.close()
    return render_template('clientes.html', clientes=lista,
        page=page, total_pages=total_pages, total=total)

@app.route('/cliente/<int:id>')
@login_required
def detalle_cliente(id):
    db = get_db()
    cursor = db.cursor(dictionary=True)

```

```

cursor.execute("SELECT * FROM clientes WHERE id = %s", (id,))
cliente = cursor.fetchone()

if not cliente:
    cursor.close()
    db.close()
    flash('Cliente no encontrado.', 'error')
    return redirect(url_for('clientes'))

cursor.execute("SELECT * FROM reparaciones WHERE cliente_id = %s ORDER BY created_at DESC", (id,))
reps = cursor.fetchall()

cursor.execute("SELECT COALESCE(SUM(precio_final), 0) as total FROM reparaciones WHERE cliente_id = %s AND
estado = 'Entregado'", (id,))
gasto = cursor.fetchone()['total']

cursor.close()
db.close()
return render_template('cliente.html', cliente=cliente, reparaciones=reps, gasto_total=gasto)

@app.route('/cliente/<int:id>/editar', methods=['GET', 'POST'])
@login_required
def editar_cliente(id):
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT * FROM clientes WHERE id = %s", (id,))
    cliente = cursor.fetchone()

    if not cliente:
        cursor.close()
        db.close()
        flash('Cliente no encontrado.', 'error')
        return redirect(url_for('clientes'))

    if request.method == 'POST':
        nombre = request.form.get('nombre', '').strip()
        telefono = request.form.get('telefono', '').strip()
        email = request.form.get('email', '').strip()

        if not nombre:
            flash('El nombre es obligatorio.', 'error')
            cursor.close()
            db.close()
            return render_template('editar_cliente.html', cliente=cliente)

        cursor.execute(
            "UPDATE clientes SET nombre = %s, telefono = %s, email = %s WHERE id = %s",
            (nombre, telefono, email, id)
        )
        db.commit()
        cursor.close()
        db.close()
        flash('Cliente actualizado correctamente.', 'success')
        return redirect(url_for('detalle_cliente', id=id))

    cursor.close()
    db.close()
    return render_template('editar_cliente.html', cliente=cliente)

@app.route('/buscar')
@login_required
def buscar():

```

```

return render_template('buscar.html')

@app.route('/api/buscar')
@login_required
def api_buscar():
    q = request.args.get('q', '').strip()
    if not q or len(q) < 2:
        return jsonify({'reparaciones': [], 'clientes': []})

    db = get_db()
    cursor = db.cursor(dictionary=True)

    cursor.execute("""
        SELECT r.*, c.nombre as cliente_nombre
        FROM reparaciones r JOIN clientes c ON r.cliente_id = c.id
        WHERE r.codigo LIKE %s OR c.nombre LIKE %s OR c.telefono LIKE %s
        ORDER BY r.created_at DESC LIMIT 10
    """, (f'%{q}%', f'%{q}%', f'%{q}%'))
    reps = cursor.fetchall()

    cursor.execute("""
        SELECT * FROM clientes
        WHERE nombre LIKE %s OR telefono LIKE %s
        LIMIT 10
    """, (f'%{q}%', f'%{q}%'))
    clientes = cursor.fetchall()

    cursor.close()
    db.close()
    return jsonify({'reparaciones': reps, 'clientes': clientes})

@app.route('/api/buscar-cliente')
@login_required
def api_buscar_cliente():
    q = request.args.get('q', '').strip()
    if not q or len(q) < 2:
        return jsonify([])

    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("""
        SELECT * FROM clientes
        WHERE nombre LIKE %s OR telefono LIKE %s OR email LIKE %s
        LIMIT 5
    """, (f'%{q}%', f'%{q}%', f'%{q}%'))
    resultados = cursor.fetchall()
    cursor.close()
    db.close()
    return jsonify(resultados)

@app.route('/pdf/<codigo>')
@login_required
def ver_pdf(codigo):
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("""
        SELECT r.*, c.nombre as nombre_cliente, c.telefono, c.email
        FROM reparaciones r
        JOIN clientes c ON r.cliente_id = c.id
        WHERE r.codigo = %s
    """, (codigo,))
    row = cursor.fetchone()
    cursor.close()

```

```

db.close()

if not row:
    flash('Reparación no encontrada.', 'error')
    return redirect(url_for('dashboard'))

datos = {
    'codigo': row['codigo'],
    'nombre_cliente': row['nombre_cliente'],
    'telefono': row.get('telefono', ''),
    'email': row.get('email', ''),
    'tipo_dispositivo': row['tipo_dispositivo'],
    'marca': row.get('marca', ''),
    'modelo': row.get('modelo', ''),
    'averia': row['averia'],
    'observaciones': row.get('observaciones', ''),
    'fecha': row['created_at']
}

pdf_dir = os.path.join(app.root_path, 'pdfs')
os.makedirs(pdf_dir, exist_ok=True)
pdf_path = os.path.join(pdf_dir, f'{codigo}.pdf')

generar_pdf(datos, pdf_path)

return send_file(pdf_path, as_attachment=False, download_name=f'{codigo}.pdf')

@app.route('/reparaciones/csv')
@login_required
def exportar_csv():
    filtro, fecha_desde, fecha_hasta, tipo_dispositivo, cliente_nombre, _, fecha_desde_str, fecha_hasta_str = \
        _get_filtros_from_request()

    db = get_db()
    cursor = db.cursor(dictionary=True)
    where_sql, params = _construir_where_filtros(filtro, fecha_desde, fecha_hasta, tipo_dispositivo,
cliente_nombre)

    cursor.execute(f"""
        SELECT r.*, c.nombre as cliente_nombre
        FROM reparaciones r JOIN clientes c ON r.cliente_id = c.id
        {where_sql}
        ORDER BY r.created_at DESC
    """, params)
    lista = cursor.fetchall()
    cursor.close()
    db.close()

    output = io.StringIO()
    output.write('') # BOM para que Excel detecte UTF-8
    writer = csv.writer(output, delimiter=';')
    writer.writerow(['Código', 'Cliente', 'Tipo dispositivo', 'Marca', 'Modelo', 'Avería', 'Estado',
'Presupuesto', 'Precio final', 'Fecha entrada'])

    for r in lista:
        fecha = r['created_at'].strftime('%d/%m/%Y') if isinstance(r['created_at'], datetime) else
str(r['created_at'])
        writer.writerow([
            r['codigo'],
            r.get('cliente_nombre', ''),
            r['tipo_dispositivo'],
            r.get('marca', ''),
            r.get('modelo', ''),
            r['averia'],
            r['estado'],

```

```

        r.get('presupuesto') or '',
        r.get('precio_final') or '',
        fecha
    ])

    output.seek(0)
    return Response(
        output.getvalue(),
        mimetype='text/csv; charset=utf-8',
        headers={'Content-Disposition': 'attachment; filename=reparaciones.csv'}
    )

@app.route('/admin')
@admin_required
def admin_panel():
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT id, nombre, email, rol, created_at FROM usuarios ORDER BY id")
    usuarios = cursor.fetchall()
    cursor.close()
    db.close()
    return render_template('admin.html', usuarios=usuarios)

@app.route('/admin/crear-usuario', methods=['POST'])
@admin_required
def crear_usuario():
    nombre = request.form.get('nombre', '').strip()
    email = request.form.get('email', '').strip()
    password = request.form.get('password', '').strip()
    rol = request.form.get('rol', 'tecnico')

    if not nombre or not email or not password:
        flash('Todos los campos son obligatorios.', 'error')
        return redirect(url_for('admin_panel'))

    if rol not in ('admin', 'tecnico'):
        rol = 'tecnico'

    db = get_db()
    cursor = db.cursor(dictionary=True)

    cursor.execute("SELECT id FROM usuarios WHERE email = %s", (email,))
    if cursor.fetchone():
        cursor.close()
        db.close()
        flash('Ya existe un usuario con ese email.', 'error')
        return redirect(url_for('admin_panel'))

    cursor.execute(
        "INSERT INTO usuarios (nombre, email, password_hash, rol) VALUES (%s, %s, %s, %s)",
        (nombre, email, generate_password_hash(password), rol)
    )
    db.commit()
    cursor.close()
    db.close()
    flash(f'Usuario "{nombre}" creado correctamente.', 'success')
    return redirect(url_for('admin_panel'))

@app.route('/admin/eliminar-usuario/<int:id>', methods=['POST'])
@admin_required
def eliminar_usuario(id):
    if id == session.get('user_id'):
        flash('No puedes eliminar tu propio usuario.', 'error')

```



```

        return redirect(url_for('admin_panel'))

db = get_db()
cursor = db.cursor(dictionary=True)
cursor.execute("SELECT id, nombre FROM usuarios WHERE id = %s", (id,))
usuario = cursor.fetchone()

if not usuario:
    cursor.close()
    db.close()
    flash('Usuario no encontrado.', 'error')
    return redirect(url_for('admin_panel'))

cursor.execute("DELETE FROM usuarios WHERE id = %s", (id,))
db.commit()
cursor.close()
db.close()
flash(f'Usuario "{usuario["nombre"]}" eliminado.', 'success')
return redirect(url_for('admin_panel'))

if __name__ == '__main__':
    app.run(debug=True)

```

A.4. Utilidades

Modulos auxiliares; generacion del PDF combinado resguardo + etiqueta recortable mediante ReportLab.

A.4.1. utilidades/pdf_generator.py

```

from reportlab.lib.pagesizes import A4
from reportlab.lib.units import cm
from reportlab.pdfgen import canvas

def _dibujar_texto_largo(c, x, y, texto, max_chars=80, paso=0.5 * cm):
    while texto:
        c.drawString(x, y, texto[:max_chars])
        texto = texto[max_chars:]
        y -= paso
    return y

def generar_pdf(datos, output_path):
    """Genera un PDF A4 con resguardo arriba y etiqueta troquelada abajo."""
    w, h = A4
    c = canvas.Canvas(output_path, pagesize=A4)

    margen = 2 * cm
    y = h - margen

    c.setFont("Helvetica-Bold", 18)
    c.drawString(margen, y, "RepairHub")
    y -= 0.6 * cm
    c.setFont("Helvetica", 9)
    c.setFillColorRGB(0.4, 0.4, 0.4)
    c.drawString(margen, y, "Servicio técnico de reparaciones – Tel: 600 000 000")
    y -= 1.5 * cm

```

```

c.setFillColorRGB(0, 0, 0)
c.setFont("Helvetica-Bold", 22)
c.drawString(margen, y, datos['codigo'])

c.setFont("Helvetica", 10)
fecha_str = datos['fecha'].strftime('%d/%m/%Y %H:%M')
c.drawRightString(w - margen, y, f"Fecha: {fecha_str}")
y -= 1.5 * cm

c.setStrokeColorRGB(0.8, 0.8, 0.8)
c.setLineWidth(0.5)
c.line(margen, y, w - margen, y)
y -= 1 * cm

c.setFont("Helvetica-Bold", 11)
c.drawString(margen, y, "DATOS DEL CLIENTE")
y -= 0.6 * cm
c.setFont("Helvetica", 10)
c.drawString(margen, y, f"Nombre: {datos['nombre_cliente']}")
y -= 0.5 * cm
contacto = []
if datos.get('telefono'):
    contacto.append(f"Tel: {datos['telefono']}")
if datos.get('email'):
    contacto.append(f"Email: {datos['email']}")
c.drawString(margen, y, " | ".join(contacto))
y -= 1 * cm

c.setFont("Helvetica-Bold", 11)
c.drawString(margen, y, "DATOS DEL DISPOSITIVO")
y -= 0.6 * cm
c.setFont("Helvetica", 10)
c.drawString(margen, y, f"Tipo: {datos['tipo_dispositivo']}      Marca: {datos.get('marca', '-')}      Modelo: {datos.get('modelo', '-')}")
y -= 0.8 * cm

c.setFont("Helvetica-Bold", 11)
c.drawString(margen, y, "AVERÍA / MOTIVO DE ENTRADA")
y -= 0.6 * cm
c.setFont("Helvetica", 10)
y = _dibujar_texto_largo(c, margen, y, datos['averia'])

if datos.get('observaciones'):
    y -= 0.3 * cm
    c.setFont("Helvetica-Bold", 11)
    c.drawString(margen, y, "OBSERVACIONES")
    y -= 0.6 * cm
    c.setFont("Helvetica", 10)
    y = _dibujar_texto_largo(c, margen, y, datos['observaciones'])

y -= 0.5 * cm
c.setFont("Helvetica", 7)
c.setFillColorRGB(0.5, 0.5, 0.5)
condiciones = [
    "CONDICIONES DE SERVICIO:",
    "1. El plazo de recogida es de 30 días desde la notificación de finalización.",
    "2. El presupuesto no incluye IVA salvo que se indique expresamente.",
    "3. RepairHub no se hace responsable de datos almacenados en el dispositivo.",
    "4. Este resguardo es necesario para la recogida del dispositivo."
]
for linea in condiciones:
    c.drawString(margen, y, linea)
    y -= 0.4 * cm

y -= 0.8 * cm
c.setFillColorRGB(0, 0, 0)
c.setFont("Helvetica", 9)
c.drawString(margen, y, "Firma del cliente: _____")

```

```

c.drawRightString(w - margen, y, f"Fecha: {datos['fecha'].strftime('%d/%m/%Y')}")

corte_y = 9 * cm
c.setDash(3, 3)
c.setStrokeColorRGB(0.6, 0.6, 0.6)
c.setLineWidth(0.5)
c.line(margen, corte_y, w - margen, corte_y)
c.setFont("Helvetica", 7)
c.setFillColorRGB(0.6, 0.6, 0.6)
c.drawCentredString(w / 2, corte_y + 0.2 * cm, "✂ Cortar por aquí – Etiqueta para el dispositivo")
c.setDash()

etiqueta_y = corte_y - 0.5 * cm
etiqueta_h = 7.5 * cm
etiqueta_x = margen

c.setStrokeColorRGB(0, 0, 0)
c.setLineWidth(1)
c.rect(etiqueta_x, etiqueta_y - etiqueta_h, w - 2 * margen, etiqueta_h)

ey = etiqueta_y - 0.8 * cm

c.setFillColorRGB(0, 0, 0)
c.setFont("Helvetica-Bold", 20)
c.drawString(etiqueta_x + 0.5 * cm, ey, datos['codigo'])

c.setFont("Helvetica-Bold", 9)
c.drawRightString(w - margen - 0.5 * cm, ey, "RepairHub")
ey -= 1 * cm

c.setFont("Helvetica", 11)
c.drawString(etiqueta_x + 0.5 * cm, ey, f"Cliente: {datos['nombre_cliente']}")
ey -= 0.7 * cm

dispositivo = f"{datos['tipo_dispositivo']} – {datos.get('marca', '')} {datos.get('modelo', '')}".strip()
c.drawString(etiqueta_x + 0.5 * cm, ey, f"Dispositivo: {dispositivo}")
ey -= 0.7 * cm

averia_corta = datos['averia'][:50] + ('...' if len(datos['averia']) > 50 else '')
c.drawString(etiqueta_x + 0.5 * cm, ey, f"Avería: {averia_corta}")
ey -= 0.7 * cm

c.setFont("Helvetica", 9)
c.setFillColorRGB(0.4, 0.4, 0.4)
c.drawString(etiqueta_x + 0.5 * cm, ey, f"Entrada: {datos['fecha'].strftime('%d/%m/%Y %H:%M')}")

c.save()
return output_path

```

A.5. Plantillas Jinja2

Capa de presentación: plantilla base, macros reutilizables y vistas para cada caso de uso.

A.5.1. plantillas/base.html

```

<!DOCTYPE html>
<html lang="es">
<head>
    <script>document.documentElement.setAttribute('data-theme', localStorage.getItem('theme'))||'light'</script>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}RepairHub{% endblock %}</title>

```

```

<link rel="icon" type="image/svg+xml" href="{{ url_for('static', filename='img/favicon.svg') }}">
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

<link
href="https://fonts.googleapis.com/css2?family=Maven+Pro:wght@400;500;600;700;800;900&family=JetBrains+Mono:wght
@400;500&display=swap" rel="stylesheet">
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
{% block head %}{% endblock %}
</head>
<body>
    <nav class="navbar">
        <div class="nav-container">
            <a href="{{ url_for('dashboard') }}" class="nav-logo">
                
                
            </a>

            <button class="hamburger" id="hamburger">
                <span></span><span></span><span></span><span></span>
            </button>

            <ul class="nav-links" id="nav-links">
                <li><a href="{{ url_for('dashboard') }}" {% if request.endpoint == 'dashboard'
}%class="active"%>Dashboard</a></li>
                <li><a href="{{ url_for('reparaciones') }}" {% if request.endpoint == 'reparaciones'
}%class="active"%>Reparaciones{% if pendientes_count %} <span class="nav-badge">{{ pendientes_count
}}</span>{% endif %}</a></li>
                <li><a href="{{ url_for('clientes') }}" {% if request.endpoint == 'clientes' %}class="active"%>
Clientes</a></li>
                <li><a href="{{ url_for('buscar') }}" {% if request.endpoint == 'buscar' %}class="active"%>
Buscar</a></li>
                {% if session.get('user_rol') == 'admin' %}
                <li><a href="{{ url_for('admin_panel') }}" {% if request.endpoint == 'admin_panel'
}%class="active"%>Admin</a></li>
                {% endif %}
            </ul>

            <div class="nav-user">
                <span class="nav-user-name">{{ session.get('user_nombre', '') }}</span>
                <a href="{{ url_for('logout') }}" class="btn btn-secondary btn-sm">Salir</a>
            </div>

            <button class="theme-toggle" id="theme-toggle" title="Cambiar tema">
                <svg class="icon-sol" width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="5"/><line x1="12" y1="1" x2="12" y2="3"/><line x1="12" y1="21" x2="12" y2="23"/><line x1="4.22" y1="4.22"
x2="5.64" y2="5.64"/><line x1="18.36" y1="18.36" x2="19.78" y2="19.78"/><line x1="1" y1="12" x2="3"
y2="12"/><line x1="21" y1="12" x2="23" y2="12"/><line x1="4.22" y1="19.78" x2="5.64" y2="18.36"/><line
x1="18.36" y1="5.64" x2="19.78" y2="4.22"/></svg>
                <svg class="icon-luna" width="18" height="18" viewBox="0 0 24 24" fill="white" stroke="white"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M21 12.79A9 9 0 1 1 11.21 3 7 7 0 0 0
21 12.79z"/></svg>
            </button>
        </div>
    </nav>

    <main class="container">
        {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
            {% for category, message in messages %}
                <div class="alert alert-{{ category }}">{{ message }}</div>
            {% endfor %}
        {% endif %}
        {% endwith %}

        {% block content %}{% endblock %}
    </main>

```

```

    </main>

<script src="{{ url_for('static', filename='js/main.js') }}"></script>
    {% block scripts %}{% endblock %}
</body>
</html>

```

A.5.2. plantillas/_macros.html

```

{% macro badge_estado(estado) -%}
<span class="badge badge-{{ estado|lower|replace(' ', '-') }}">{{ estado }}</span>
{%- endmacro %}

```

A.5.3. plantillas/login.html

```

<!DOCTYPE html>
<html lang="es">
<head>
    <script>document.documentElement.setAttribute('data-theme',localStorage.getItem('theme')||'light')</script>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Iniciar sesión – RepairHub</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Maven+Pro:wght@400;500;600;700;800;900&family=JetBrains+Mono:wght
@400;500&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
    <div class="login-page">
        <div class="login-card">
            <div class="login-logo">
                
                
            </div>

            <h1>Iniciar sesión</h1>
            <p class="login-sub">Acceso al panel de gestión</p>

            {% with messages = get_flashed_messages(with_categories=true) %}
                {% if messages %}
                    {% for category, message in messages %}
                        <div class="alert alert-{{ category }}">{{ message }}</div>
                    {% endfor %}
                {% endif %}
            {% endwith %}

            <form method="POST" action="{{ url_for('login') }}">
                <div class="form-group">
                    <label for="email">Email</label>
                    <input type="email" id="email" name="email" required placeholder="tu@email.com" autofocus>
                </div>

                <div class="form-group">
                    <label for="password">Contraseña</label>
                    <input type="password" id="password" name="password" required placeholder="Contraseña">
                </div>

                <button type="submit" class="btn btn-primary btn-lg" style="width:100%">Entrar</button>
            </form>
        </div>
    </div>

```

```

        <button class="theme-toggle login-theme-toggle" id="theme-toggle" title="Cambiar tema">
            <svg class="icon-sol" width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12" r="5"/><line x1="12"
y1="1" x2="12" y2="3"/><line x1="12" y1="21" x2="12" y2="23"/><line x1="4.22" y1="4.22" x2="5.64"
y2="5.64"/><line x1="18.36" y1="18.36" x2="19.78" y2="19.78"/><line x1="1" y1="12" x2="3" y2="12"/><line x1="21"
y1="12" x2="23" y2="12"/><line x1="4.22" y1="19.78" x2="5.64" y2="18.36"/><line x1="18.36" y1="5.64" x2="19.78"
y2="4.22"/></svg>
            <svg class="icon-luna" width="18" height="18" viewBox="0 0 24 24" fill="white" stroke="white"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M21 12.79A9 9 0 1 1 11.21 3 7 7 0 0 0
21 12.79z"/></svg>
        </button>
    </div>

    <script src="{ { url_for('static', filename='js/main.js') } }"></script>
</body>
</html>

```

A.5.4. plantillas/dashboard.html

```

{% extends "base.html" %}
{% from "_macros.html" import badge_estado %}
{% block title %}Panel de control – RepairHub{% endblock %}

{% block head %}
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
{% endblock %}

{% block content %}

<header class="page-intro">
    <div class="kicker">
        <span class="kicker-dot"></span>
        <span>Panel de control</span>
        <span class="kicker-sep">·</span>
        <span id="kickerDate"></span>
        <span class="kicker-sep">·</span>
        <span>RepairHub</span>
    </div>

    <div class="page-intro-row">
        <h1 class="page-title">
            Resumen del <span class="accent">taller</span>
        </h1>
        <a href="{ { url_for('nueva_entrada') } }" class="btn btn-primary btn-lg btn-nueva-entrada">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"
stroke-linecap="round" stroke-linejoin="round"><line x1="12" y1="5" x2="12" y2="19"/><line x1="5" y1="12"
x2="19" y2="12"/></svg>
            Nueva entrada
        </a>
    </div>

    <p class="page-intro-sub">
        Estado operativo, distribución de cargas y actividad reciente del periodo en curso.
    </p>
</header>

<div class="dashboard-grid">

    <div class="dashboard-main">

        <section class="kpi-grid" aria-label="Indicadores clave">
            <article class="kpi">

```

```

        <div class="kpi-head">
            <span class="kpi-label">En curso</span>
            <span class="kpi-index">01</span>
        </div>
        <div class="kpi-value">{{ pendientes }}</div>
        <div class="kpi-foot">Reparaciones activas pendientes de entrega</div>
    </article>

    <article class="kpi">
        <div class="kpi-head">
            <span class="kpi-label">Este mes</span>
            <span class="kpi-index">02</span>
        </div>
        <div class="kpi-value">{{ este_mes }}</div>
        <div class="kpi-foot">Entradas registradas en el periodo actual</div>
    </article>

    <article class="kpi">
        <div class="kpi-head">
            <span class="kpi-label">Ingresos / mes</span>
            <span class="kpi-index">03</span>
        </div>
        <div class="kpi-value">
            {{ "%.0f"|format(ingresos_mes) }}<span class="unit">€</span>
        </div>
        <div class="kpi-foot">Facturado en reparaciones entregadas</div>
    </article>

    <article class="kpi">
        <div class="kpi-head">
            <span class="kpi-label">Tiempo medio</span>
            <span class="kpi-index">04</span>
        </div>
        <div class="kpi-value">
            {{ tiempo_medio }}<span class="unit">días</span>
        </div>
        <div class="kpi-foot">Desde entrada hasta entrega al cliente</div>
    </article>
</section>

<section class="panel">
    <header class="panel-head">
        <div>
            <span class="panel-eyebrow"><span class="num">§ 01</span> – Flujo operativo</span>
            <h2 class="panel-title">Reparaciones por estado</h2>
        </div>
        <a href="{{ url_for('reparaciones') }}" class="link-arrow">Ver todas →</a>
    </header>

    <div class="pipeline" id="pipeline"></div>
</section>

<div class="panel-row">
    <section class="panel">
        <header class="panel-head">
            <div>
                <span class="panel-eyebrow"><span class="num">§ 02</span> – Distribución</span>
                <h2 class="panel-title">Tipos de dispositivo</h2>
            </div>
        </header>
        <div class="chart-wrap">
            <canvas id="chartTipos"></canvas>
        </div>
    </section>

```

```

        <section class="panel">
            <header class="panel-head">
                <div>
                    <span class="panel-eyebrow"><span class="num">§ 03</span> – Carga de trabajo</span>
                    <h2 class="panel-title">Reparaciones por estado</h2>
                </div>
            </header>
            <div class="chart-wrap">
                <canvas id="chartEstados"></canvas>
            </div>
        </section>
    </div>

</div>

<aside class="dashboard-aside">
    <section class="panel panel-feed">
        <header class="panel-head">
            <div>
                <span class="panel-eyebrow"><span class="num">§ 04</span> – Actividad</span>
                <h2 class="panel-title">Últimas entradas</h2>
            </div>
            <a href="{ { url_for('reparaciones') } }" class="link-arrow">Todas →</a>
        </header>

        <ul class="feed-list">
            {% for rep in ultimas %}
            <li class="feed-item">
                <a href="/reparacion/{ { rep.codigo } }" class="feed-link">
                    <div class="feed-meta">
                        <span class="feed-code mono">{ { rep.codigo } }</span>
                        <span class="feed-date mono">{ { rep.created_at.strftime('%d.%m') } }</span>
                    </div>
                    <div class="feed-client">{ { rep.cliente_nombre } }</div>
                    <div class="feed-device">
                        { { rep.tipo_dispositivo } } <span class="text-muted">• { { rep.marca } } { { rep.modelo
or ' ' } }</span>
                    </div>
                    <div class="feed-foot">
                        { { badge_estado(rep.estado) } }
                    </div>
                </a>
            </li>
            {% endfor %}

            {% if ultimas|length == 0 %}
            <li class="empty-state">Sin actividad reciente.</li>
            {% endif %}
        </ul>
    </section>
</aside>

</div>

{% endblock %}

{% block scripts %}
<script>
(function() {
    const meses = ['ene', 'feb', 'mar', 'abr', 'may', 'jun', 'jul', 'ago', 'sep', 'oct', 'nov', 'dic'];
    const d = new Date();
    const fecha = ` ${String(d.getDate()).padStart(2, '0')} ${meses[d.getMonth()]} ${d.getFullYear()}`;
    document.getElementById('kickerDate').textContent = fecha;
})();

```



```

const reparaciones = ({ reparaciones | tojson });

const estadosFlujo = [
  'Recibido',
  'Diagnosticado',
  'Presupuesto enviado',
  'Presupuesto aceptado',
  'Reparando',
  'Esperando pieza',
  'Listo',
  'Entregado'
];

const conteoEstados = {};
reparaciones.forEach(r => {
  conteoEstados[r.estado] = (conteoEstados[r.estado] || 0) + 1;
});

const pipeline = document.getElementById('pipeline');
estadosFlujo.forEach((estado, idx) => {
  const count = conteoEstados[estado] || 0;
  const cell = document.createElement('div');
  cell.className = 'pipeline-cell' + (count > 0 ? ' has-count' : ' empty');
  cell.innerHTML = `
    <span class="pipeline-step">${String(idx + 1).padStart(2, '0')}</span>
    <span class="pipeline-label">${estado}</span>
    <span class="pipeline-count">${count}</span>
  `;
  pipeline.appendChild(cell);
});

const isDark = document.documentElement.getAttribute('data-theme') === 'dark';
const inkPrimary = isDark ? '#EFEDE5' : '#111110';
const inkMuted = isDark ? '#8A867B' : '#6E6B62';
const ruleColor = isDark ? '#26251F' : '#DCD7CB';
const accent = isDark ? '#7AA0D9' : '#1B3A6F';

const paletaCalida = isDark
  ? ['#7AA0D9', '#A8C2E2', '#C9DBEC', '#9AAA85', '#C5C2B8', '#8A867B']
  : ['#1B3A6F', '#3F6CA8', '#7A95C9', '#5B6B7A', '#8A867B', '#C9C5B8'];

Chart.defaults.font.family = "'Maven Pro', system-ui, sans-serif";
Chart.defaults.font.size = 12;
Chart.defaults.color = inkMuted;
Chart.defaults.borderColor = ruleColor;

const conteoTipos = {};
reparaciones.forEach(r => {
  conteoTipos[r.tipo_dispositivo] = (conteoTipos[r.tipo_dispositivo] || 0) + 1;
});

new Chart(document.getElementById('chartTipos'), {
  type: 'doughnut',
  data: {
    labels: Object.keys(conteoTipos),
    datasets: [{
      data: Object.values(conteoTipos),
      backgroundColor: paletaCalida.slice(0, Object.keys(conteoTipos).length),
      borderColor: isDark ? '#141413' : '#FBFAF6',
      borderWidth: 3,
      hoverOffset: 8
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    cutout: '62%',
    plugins: {

```

```

        legend: {
            position: 'right',
            labels: {
                boxWidth: 12,
                boxHeight: 12,
                padding: 14,
                font: { size: 13, family: "'Maven Pro', sans-serif", weight: 500 },
                color: inkPrimary,
                usePointStyle: true,
                pointStyle: 'rect'
            }
        },
        tooltip: {
            backgroundColor: inkPrimary,
            titleColor: isDark ? '#0E0E0D' : '#FBFAF6',
            bodyColor: isDark ? '#0E0E0D' : '#FBFAF6',
            titleFont: { family: "'JetBrains Mono', monospace", size: 11, weight: '500' },
            bodyFont: { family: "'Maven Pro', sans-serif", size: 13 },
            padding: 11,
            cornerRadius: 4,
            displayColors: false
        }
    }
});

new Chart(document.getElementById('chartEstados'), {
    type: 'bar',
    data: {
        labels: Object.keys(conteoEstados),
        datasets: [{
            data: Object.values(conteoEstados),
            backgroundColor: accent,
            hoverBackgroundColor: isDark ? '#A8C2E2' : '#142D54',
            borderRadius: 3,
            barThickness: 22
        }]
    },
    options: {
        indexAxis: 'y',
        responsive: true,
        maintainAspectRatio: false,
        plugins: {
            legend: { display: false },
            tooltip: {
                backgroundColor: inkPrimary,
                titleColor: isDark ? '#0E0E0D' : '#FBFAF6',
                bodyColor: isDark ? '#0E0E0D' : '#FBFAF6',
                titleFont: { family: "'JetBrains Mono', monospace", size: 11 },
                bodyFont: { family: "'Maven Pro', sans-serif", size: 13 },
                padding: 11,
                cornerRadius: 4,
                displayColors: false
            }
        },
        scales: {
            x: {
                beginAtZero: true,
                ticks: { stepSize: 1, color: inkMuted, font: { family: "'JetBrains Mono', monospace", size: 11 } },
                grid: { color: ruleColor, drawTicks: false },
                border: { display: false }
            },
            y: {
                ticks: { color: inkPrimary, font: { family: "'Maven Pro', sans-serif", size: 12.5, weight: 500 } },
                grid: { display: false },
                border: { color: ruleColor }
            }
        }
    }
});

```

```
});
</script>
{% endblock %}
```

A.5.5. plantillas/admin.html

```
{% extends 'base.html' %}

{% block title %}Admin – RepairHub{% endblock %}

{% block content %}
<h1>Panel de administración</h1>

<div class="admin-grid">
  <div class="card">
    <h3>Crear nuevo usuario</h3>
    <form method="POST" action="{{ url_for('crear_usuario') }}">
      <div class="form-group">
        <label for="nombre">Nombre</label>
        <input type="text" id="nombre" name="nombre" required placeholder="Nombre del técnico">
      </div>
      <div class="form-group">
        <label for="email">Email</label>
        <input type="email" id="email" name="email" required placeholder="email@repairhub.com">
      </div>
      <div class="form-group">
        <label for="password">Contraseña</label>
        <input type="password" id="password" name="password" required placeholder="Contraseña"
minlength="6">
      </div>
      <div class="form-group">
        <label for="rol">Rol</label>
        <select id="rol" name="rol">
          <option value="tecnico">Técnico</option>
          <option value="admin">Administrador</option>
        </select>
      </div>
      <button type="submit" class="btn btn-primary">Crear usuario</button>
    </form>
  </div>

  <div class="card">
    <h3>Usuarios registrados</h3>
    {% if usuarios %}
    <div class="table-responsive" style="border:none;box-shadow:none;">
      <table>
        <thead>
          <tr>
            <th>Nombre</th>
            <th>Email</th>
            <th>Rol</th>
            <th></th>
          </tr>
        </thead>
        <tbody>
          {% for u in usuarios %}
          <tr>
            <td>{{ u.nombre }}</td>
            <td>{{ u.email }}</td>
            <td><span class="badge badge-rol-{{ u.rol }}">{{ u.rol|capitalize }}</span></td>
            <td>
              {% if u.id != session.get('user_id') %}
              <form method="POST" action="{{ url_for('eliminar_usuario', id=u.id) }}"
class="form-inline" onsubmit="return confirm('¿Eliminar al usuario {{ u.nombre }}?')">
                <button type="submit" class="btn btn-secondary btn-sm
btn-danger-text">Eliminar</button>
              </form>
            </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
    {% else %}
    <p>No hay usuarios registrados.</p>
    {% endif %}
  </div>
</div>
{% endblock %}
```

```

                {% else %}
                <span class="text-muted" style="font-size:0.8rem">Tú</span>
                {% endif %}
            </td>
        </tr>
    </tbody>
</table>
</div>
{% else %}
<p class="empty-state">No hay usuarios registrados.</p>
{% endif %}
</div>
</div>
{% endblock %}

```

A.5.6. plantillas/clientes.html

```

{% extends "base.html" %}
{% block title %}Clientes – RepairHub{% endblock %}

{% block content %}
<div class="page-header">
    <h1>Clientes <span class="text-muted">({{ total }})</span></h1>
</div>

<div class="table-responsive">
    <table>
        <thead>
            <tr>
                <th>Nombre</th>
                <th>Teléfono</th>
                <th>Email</th>
                <th>Reparaciones</th>
                <th>Alta</th>
            </tr>
        </thead>
        <tbody>
            {% for c in clientes %}
            <tr>
                <td><a href="/cliente/{{ c.id }}">{{ c.nombre }}</a></td>
                <td>{{ c.telefono or '-' }}</td>
                <td>{{ c.email or '-' }}</td>
                <td>{{ c.n_reparaciones }}</td>
                <td>{{ c.created_at.strftime('%d/%m/%Y') }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>

{% if total_pages > 1 %}
<nav class="paginacion">
    {% if page > 1 %}
    <a href="{{ url_for('clientes', page=page-1) }}" class="btn btn-secondary btn-sm">&laquo; Anterior</a>
    {% endif %}

    <span class="paginacion-info">Página {{ page }} de {{ total_pages }}</span>

    {% if page < total_pages %}
    <a href="{{ url_for('clientes', page=page+1) }}" class="btn btn-secondary btn-sm">Siguiete &raquo;</a>
    {% endif %}
</nav>
{% endif %}
{% endblock %}

```

A.5.7. plantillas/cliente.html

```
{% extends "base.html" %}
{% from "_macros.html" import badge_estado %}
{% block title %}{{ cliente.nombre }} – RepairHub{% endblock %}

{% block content %}
<div class="page-header">
  <h1>{{ cliente.nombre }}</h1>
  <a href="{{ url_for('editar_cliente', id=cliente.id) }}" class="btn btn-secondary">Editar</a>
</div>

<div class="card">
  <p>Teléfono: {{ cliente.telefono or '-' }}</p>
  <p>Email: {{ cliente.email or '-' }}</p>
  <p>Alta: {{ cliente.created_at.strftime('%d/%m/%Y') }}</p>
  <p>Gasto total: {{ "%.2f"|format(gasto_total) }} €</p>
  <p>Total reparaciones: {{ reparaciones|length }}</p>
</div>

<h2>Historial de reparaciones</h2>
<div class="table-responsive">
  <table>
    <thead>
      <tr><th>Código</th><th>Dispositivo</th><th>Avería</th><th>Estado</th><th>Fecha</th></tr>
    </thead>
    <tbody>
      {% for rep in reparaciones %}
      <tr>
        <td><a href="/reparacion/{{ rep.codigo }}">{{ rep.codigo }}</a></td>
        <td>{{ rep.tipo_dispositivo }} {{ rep.marca }}</td>
        <td>{{ rep.averia[:40] }}...</td>
        <td>{{ badge_estado(rep.estado) }}</td>
        <td>{{ rep.created_at.strftime('%d/%m/%Y') }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endblock %}
```

A.5.8. plantillas/editar_cliente.html

```
{% extends "base.html" %}
{% block title %}Editar {{ cliente.nombre }} – RepairHub{% endblock %}

{% block content %}
<h1>Editar cliente</h1>

<div class="card">
  <form method="POST" class="form-grid">
    <div class="form-group">
      <label for="nombre">Nombre *</label>
      <input type="text" id="nombre" name="nombre" value="{{ cliente.nombre }}" required>
    </div>

    <div class="form-group">
      <label for="telefono">Teléfono</label>
      <input type="tel" id="telefono" name="telefono" value="{{ cliente.telefono or '' }}">
    </div>

    <div class="form-group">
      <label for="email">Email</label>
      <input type="email" id="email" name="email" value="{{ cliente.email or '' }}">
    </div>
  </form>
</div>
```

```

        <div class="form-actions">
            <button type="submit" class="btn btn-primary">Guardar cambios</button>
            <a href="{{ url_for('detalle_cliente', id=cliente.id) }}" class="btn btn-secondary">Cancelar</a>
        </div>
    </form>
</div>
{% endblock %}

```

A.5.9. plantillas/reparaciones.html

```

{% extends "base.html" %}
{% from "_macros.html" import badge_estado %}
{% block title %}Reparaciones – RepairHub{% endblock %}

{% block content %}
<div class="page-header">
    <h1>Reparaciones <span class="text-muted">({{ total }})</span></h1>
    <a href="{{ url_for('exportar_csv', estado=filtro_actual, fecha_desde=fecha_desde, fecha_hasta=fecha_hasta,
tipo_dispositivo=tipo_dispositivo, cliente=cliente) }}" class="btn btn-secondary btn-sm">
        <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"
stroke-linecap="round" stroke-linejoin="round"><path d="M21 15v4a2 2 0 01-2 2H5a2 2 0 01-2-2v-4"/><polyline
points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
        Exportar CSV
    </a>
</div>

<div class="filtros">
    {% set estados = ['todos', 'Recibido', 'Diagnosticado', 'Presupuesto enviado', 'Presupuesto aceptado',
'Reparando', 'Esperando pieza', 'Listo', 'Entregado'] %}
    {% for est in estados %}
        <a href="{{ url_for('reparaciones', estado=est, fecha_desde=fecha_desde, fecha_hasta=fecha_hasta,
tipo_dispositivo=tipo_dispositivo, cliente=cliente) }}"
            class="btn {% if filtro_actual == est %}btn-primary{% else %}btn-secondary{% endif %}">
            {{ est|capitalize }}
        </a>
    {% endfor %}
</div>

<div class="filtros-avanzados card">
    <form method="GET" action="{{ url_for('reparaciones') }}" class="filtros-form">
        <input type="hidden" name="estado" value="{{ filtro_actual }}">
        <div class="form-group">
            <label>Desde</label>
            <input type="date" name="fecha_desde" value="{{ fecha_desde }}">
        </div>
        <div class="form-group">
            <label>Hasta</label>
            <input type="date" name="fecha_hasta" value="{{ fecha_hasta }}">
        </div>
        <div class="form-group">
            <label>Tipo dispositivo</label>
            <select name="tipo_dispositivo">
                <option value="">Todos</option>
                {% for tipo in tipos_dispositivo %}
                    <option value="{{ tipo }}" {% if tipo_dispositivo == tipo %}selected{% endif %}>{{ tipo
}}</option>
                {% endfor %}
            </select>
        </div>
        <div class="form-group">
            <label>Cliente</label>
            <input type="text" name="cliente" value="{{ cliente }}" placeholder="Nombre del cliente">
        </div>
        <div class="filtros-form-actions">
            <button type="submit" class="btn btn-primary btn-sm">Filtrar</button>
            <a href="{{ url_for('reparaciones') }}" class="btn btn-secondary btn-sm">Limpiar</a>
        </div>
    </form>
</div>

```

```

        </div>
    </form>
</div>

<div class="table-responsive">
    <table>
        <thead>
            <tr>
                <th>Código</th>
                <th>Cliente</th>
                <th>Dispositivo</th>
                <th>Avería</th>
                <th>Estado</th>
                <th>Fecha</th>
            </tr>
        </thead>
        <tbody>
            {% for rep in reparaciones %}
            <tr>
                <td><a href="/reparacion/{{ rep.codigo }}">{{ rep.codigo }}</a></td>
                <td>{{ rep.cliente_nombre }}</td>
                <td>{{ rep.tipo_dispositivo }} {{ rep.marca }}</td>
                <td>{{ rep.averia[:40] }}{% if rep.averia|length > 40 %}...{% endif %}</td>
                <td>{{ badge_estado(rep.estado) }}</td>
                <td>{{ rep.created_at.strftime('%d/%m/%Y') }}</td>
            </tr>
            {% endfor %}

            {% if reparaciones|length == 0 %}
            <tr><td colspan="6" class="empty-state">No hay reparaciones</td></tr>
            {% endif %}
        </tbody>
    </table>
</div>

{% if total_pages > 1 %}
<nav class="paginacion">
    {% if page > 1 %}
        <a href="{{ url_for('reparaciones', page=page-1, estado=filtro_actual, fecha_desde=fecha_desde,
fecha_hasta=fecha_hasta, tipo_dispositivo=tipo_dispositivo, cliente=cliente) }}" class="btn btn-secondary
btn-sm">&laquo; Anterior</a>
        {% endif %}

        <span class="paginacion-info">Página {{ page }} de {{ total_pages }}</span>

        {% if page < total_pages %}
            <a href="{{ url_for('reparaciones', page=page+1, estado=filtro_actual, fecha_desde=fecha_desde,
fecha_hasta=fecha_hasta, tipo_dispositivo=tipo_dispositivo, cliente=cliente) }}" class="btn btn-secondary
btn-sm">Siguiente &raquo;</a>
        {% endif %}
    </nav>
{% endif %}
{% endblock %}

```

A.5.10. plantillas/reparacion.html

```

{% extends "base.html" %}
{% from "_macros.html" import badge_estado %}
{% block title %}{{ rep.codigo }} – RepairHub{% endblock %}

{% block content %}
<h1>{{ rep.codigo }}</h1>
{{ badge_estado(rep.estado) }}

<div class="detalle-grid">
    <div class="card">

```

```

<h3>Cliente</h3>
<p><strong>{{ rep.cliente_nombre }}</strong></p>
{% if rep.cliente_telefono %}<p>Tel: {{ rep.cliente_telefono }}</p>{% endif %}
{% if rep.cliente_email %}<p>Email: {{ rep.cliente_email }}</p>{% endif %}
</div>

<div class="card">
<h3>Dispositivo</h3>
<p>{{ rep.tipo_dispositivo }} – {{ rep.marca }} {{ rep.modelo }}</p>
<p><strong>Avería:</strong> {{ rep.averia }}</p>
{% if rep.observaciones %}<p><strong>Obs:</strong> {{ rep.observaciones }}</p>{% endif %}
</div>

<div class="card">
<h3>Presupuesto / Precio</h3>
<p>Presupuesto: {{ rep.presupuesto if rep.presupuesto else '-' }} €</p>
<p>Precio final: {{ rep.precio_final if rep.precio_final else '-' }} €</p>

{% if rep.estado == 'Diagnosticado' %}
<form method="POST" action="/reparacion/{{ rep.codigo }}/presupuesto">
<div class="form-group">
<label>Importe presupuesto (€)</label>
<input type="number" name="presupuesto" step="0.01" min="0" required>
</div>
<button type="submit" class="btn btn-primary">Enviar presupuesto</button>
</form>
{% endif %}

{% if rep.estado == 'Listo' and not rep.precio_final %}
<form method="POST" action="/reparacion/{{ rep.codigo }}/precio-final">
<div class="form-group">
<label>Precio final (€)</label>
<input type="number" name="precio_final" step="0.01" min="0"
value="{{ rep.presupuesto or '' }}" required>
</div>
<button type="submit" class="btn btn-primary">Guardar precio</button>
</form>
{% endif %}
</div>
</div>

{% if estados_posibles %}
<div class="acciones-estado">
<h3>Cambiar estado</h3>
<form method="POST" action="/reparacion/{{ rep.codigo }}/cambiar-estado" class="form-estado-inline">
<div class="form-group">
<select name="nuevo_estado" required>
<option value="" disabled selected>Seleccionar estado...</option>
{% for estado in estados_posibles %}
<option value="{{ estado }}">{{ estado }}</option>
{% endfor %}
</select>
</div>
<button type="submit" class="btn btn-primary">Cambiar estado</button>
</form>
</div>
{% endif %}

<div class="acciones-reparacion">
<a href="/pdf/{{ rep.codigo }}" class="btn btn-secondary" target="_blank">Reimprimir PDF</a>
<a href="{{ url_for('editar_reparacion', codigo=rep.codigo) }}" class="btn btn-secondary">Editar datos</a>
<button type="button" class="btn btn-danger"
onclick="document.getElementById('modal-eliminar').style.display='flex'">Eliminar</button>
</div>

<div id="modal-eliminar" class="modal-overlay" style="display:none">
<div class="modal">
<p class="modal-code">{{ rep.codigo }}</p>

```



```

        <p class="modal-msg">¿Seguro que quieres eliminar esta reparación? Esta acción no se puede deshacer.</p>
        <div class="modal-actions">
            <form method="POST" action="{{ url_for('eliminar_reparacion', codigo=rep.codigo) }}">
                <button type="submit" class="btn btn-danger">Sí, eliminar</button>
            </form>
            <button type="button" class="modal-close"
onclick="document.getElementById('modal-eliminar').style.display='none'">Cancelar</button>
        </div>
    </div>
</div>

<h2>Historial</h2>
<div class="table-responsive">
    <table>
        <thead><tr><th>Estado</th><th>Técnico</th><th>Fecha</th></tr></thead>
        <tbody>
            {% for h in historial %}
            <tr>
                <td>{{ badge_estado(h.estado) }}</td>
                <td>{{ h.tecnico or '-' }}</td>
                <td>{{ h.fecha.strftime('%d/%m/%Y %H:%M') }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
{% endblock %}

```

A.5.11. plantillas/editar_reparacion.html

```

{% extends "base.html" %}
{% block title %}Editar {{ rep.codigo }} – RepairHub{% endblock %}

{% block content %}
<h1>Editar {{ rep.codigo }}</h1>

<div class="card">
    <form method="POST">
        <div class="form-group">
            <label for="tipo_dispositivo">Tipo de dispositivo *</label>
            <select name="tipo_dispositivo" id="tipo_dispositivo" required>
                {% for tipo in ['Móvil', 'Portátil', 'Sobremesa', 'Tablet', 'Consola', 'Otro'] %}
                <option value="{{ tipo }}" {% if rep.tipo_dispositivo == tipo %>selected{% endif %}>{{ tipo }}</option>
                {% endfor %}
            </select>
        </div>

        <div class="form-group">
            <label for="marca">Marca</label>
            <input type="text" name="marca" id="marca" value="{{ rep.marca }}">
        </div>

        <div class="form-group">
            <label for="modelo">Modelo</label>
            <input type="text" name="modelo" id="modelo" value="{{ rep.modelo }}">
        </div>

        <div class="form-group">
            <label for="averia">Avería *</label>
            <textarea name="averia" id="averia" rows="3" required>{{ rep.averia }}</textarea>
        </div>

        <div class="form-group">
            <label for="observaciones">Observaciones</label>
            <textarea name="observaciones" id="observaciones" rows="3">{{ rep.observaciones or '' }}</textarea>
        </div>
    </form>
</div>

```

```

    </div>

    <div class="form-actions">
        <button type="submit" class="btn btn-primary">Guardar cambios</button>
        <a href="{{ url_for('detalle_reparacion', codigo=rep.codigo) }}" class="btn
btn-secondary">Cancelar</a>
    </div>
</form>
</div>
{% endblock %}

```

A.5.12. plantillas/nueva_entrada.html

```

{% extends "base.html" %}
{% block title %}Nueva entrada – RepairHub{% endblock %}

{% block content %}
<h1>Nueva entrada</h1>

{% if request.args.get('pdf') %}
{% set pdf_codigo = request.args.get('pdf') %}
<div id="modal-overlay" class="modal-overlay">
    <div class="modal">
        <p class="modal-code">{{ pdf_codigo }}</p>
        <p class="modal-msg">Creado correctamente.</p>
        <div class="modal-actions">
            <a href="{{ url_for('ver_pdf', codigo=pdf_codigo) }}" target="_blank" class="btn btn-primary">
                <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0
1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
                Descargar {{ pdf_codigo }}.pdf
            </a>
            <button type="button" class="btn btn-secondary" onclick="var w=window.open('{{ url_for('ver_pdf',
codigo=pdf_codigo) }}', '_blank');if(w)setTimeout(function(){w.print()},1000);">
                <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polyline points="6 9 6 2 18 2 18 9"/><path
d="M6 18H4a2 2 0 0 1-2-2v-5a2 2 0 0 1 2-2h16a2 2 0 0 1 2 2v5a2 2 0 0 1-2 2h-2"/><rect x="6" y="14" width="12"
height="8"/></svg>
                Imprimir resguardo
            </button>
        </div>
    </div>
    <button type="button" class="btn modal-close"
onclick="document.getElementById('modal-overlay').remove()">Cerrar</button>
</div>
</div>
{% endif %}

<form method="POST" action="{{ url_for('nueva_entrada') }}" class="form-nueva-entrada">

    <h2>Datos del cliente</h2>
    <div class="form-row">
        <div class="form-group">
            <label for="nombre">Nombre *</label>
            <input type="text" id="nombre" name="nombre" required placeholder="Nombre del cliente">
        </div>
        <div class="form-group">
            <label for="telefono">Teléfono</label>
            <input type="tel" id="telefono" name="telefono" placeholder="612 345 678">
        </div>
        <div class="form-group">
            <label for="email">Email</label>
            <input type="email" id="email" name="email" placeholder="cliente@email.com">
        </div>
    </div>
    <p class="form-hint">* Al menos un dato de contacto (teléfono o email)</p>

    <h2>Datos del dispositivo</h2>

```

```

<div class="form-row">
  <div class="form-group">
    <label for="tipo_dispositivo">Tipo de dispositivo *</label>
    <select id="tipo_dispositivo" name="tipo_dispositivo" required>
      <option value="">Seleccionar...</option>
      <option value="Móvil">Móvil</option>
      <option value="Tablet">Tablet</option>
      <option value="Portátil">Portátil</option>
      <option value="Ordenador">Ordenador</option>
      <option value="Consola">Consola</option>
      <option value="Otro">Otro</option>
    </select>
  </div>
  <div class="form-group">
    <label for="marca">Marca</label>
    <input type="text" id="marca" name="marca" placeholder="Samsung, Apple, HP...">
  </div>
  <div class="form-group">
    <label for="modelo">Modelo</label>
    <input type="text" id="modelo" name="modelo" placeholder="Galaxy S23, iPhone 14...">
  </div>
</div>

<div class="form-group">
  <label for="averia">Descripción de la avería *</label>
  <textarea id="averia" name="averia" required placeholder="Describir el problema del
dispositivo..."></textarea>
</div>

<div class="form-group">
  <label for="observaciones">Observaciones / Estado estético</label>
  <textarea id="observaciones" name="observaciones" placeholder="Golpes, arañazos, accesorios
entregados..."></textarea>
</div>

<button type="submit" class="btn btn-primary">Dar entrada y generar resguardo</button>
</form>
{% endblock %}

{% block scripts %}
{% if request.args.get('pdf') %}
<script>
  document.addEventListener('keydown', function(e) {
    if (e.key === 'Escape') {
      var overlay = document.getElementById('modal-overlay');
      if (overlay) overlay.remove();
    }
  });
  document.getElementById('modal-overlay').addEventListener('click', function(e) {
    if (e.target === this) this.remove();
  });
</script>
{% endif %}
{% endblock %}

```

A.5.13. plantillas/buscar.html

```

{% extends "base.html" %}
{% block title %}Buscar – RepairHub{% endblock %}

{% block content %}
<h1>Buscar</h1>

<div class="form-group">
  <input type="text" id="campo-busqueda" placeholder="Buscar por código, nombre o teléfono..." autofocus>
</div>

```

```

<div id="resultados-busqueda"></div>
{% endblock %}

{% block scripts %}
<script>
const campo = document.getElementById('campo-busqueda');
const resultados = document.getElementById('resultados-busqueda');
let timeout;

campo.addEventListener('input', () => {
  clearTimeout(timeout);
  const q = campo.value.trim();
  if (q.length < 2) { resultados.innerHTML = ''; return; }

  timeout = setTimeout(() => {
    fetch(`/api/buscar?q=${encodeURIComponent(q)}`)
      .then(res => res.json())
      .then(data => {
        let html = '';

        if (data.reparaciones.length > 0) {
          html += '<h2>Reparaciones</h2><div class="table-responsive"><table>';
          html += '<thead><tr><th>Código</th><th>Cliente</th><th>Dispositivo</th><th>Estado</th></tr></thead><tbody>';
          data.reparaciones.forEach(r => {
            html += '<tr>';
            html += '<td><a href="/reparacion/${r.codigo}">${r.codigo}</a></td>';
            html += '<td>${r.cliente_nombre || ''}</td>';
            html += '<td>${r.tipo_dispositivo} ${r.marca || ''}</td>';
            html += '<td>${r.estado}</td>';
            html += '</tr>';
          });
          html += '</tbody></table></div>';
        }

        if (data.clientes.length > 0) {
          html += '<h2>Clientes</h2><div class="table-responsive"><table>';
          html += '<thead><tr><th>Nombre</th><th>Teléfono</th><th>Email</th></tr></thead><tbody>';
          data.clientes.forEach(c => {
            html += '<tr>';
            html += '<td><a href="/cliente/${c.id}">${c.nombre}</a></td>';
            html += '<td>${c.telefono || ''}</td>';
            html += '<td>${c.email || ''}</td>';
            html += '</tr>';
          });
          html += '</tbody></table></div>';
        }

        if (!html) html = '<p style="color: var(--text-muted);">Sin resultados</p>';
        resultados.innerHTML = html;
      });
  }, 300);
});
</script>
{% endblock %}

```

A.6. Recursos staticos

Hoja de estilos con el sistema de tokens del tema dual claro/oscurο y el script JavaScript de cliente.

A.6.1. staticos/css/style.css

```
/* === TOKENS DE TEMA === */
```

```

:root {
  /* superficies */
  --bg-canvas: #F4F2ED; /* papel cremoso */
  --bg-surface: #FBFAF6; /* tarjetas */
  --bg-elevated: #FFFFFF; /* zonas activas */
  --bg-sunken: #ECE9E2; /* tabla header, código */

  /* tinta */
  --ink-1: #111110; /* texto principal */
  --ink-2: #2F2E2A; /* texto secundario */
  --ink-3: #6E6B62; /* texto muted */
  --ink-4: #A29E92; /* placeholders */
  --ink-5: #C9C5B8; /* dimmed */

  /* trazos */
  --rule: #DCD7CB; /* hairline */
  --rule-strong: #B3AE9F; /* línea destacada */
  --rule-bold: #111110; /* línea editorial */

  /* acento – azul blueprint / plano técnico */
  --accent: #1B3A6F;
  --accent-deep: #142D54;
  --accent-soft: #E6ECF5;
  --accent-text: #FBFAF6;
  --ink-button: #111110; /* botón principal: tinta */

  /* tonos secundarios coordinados con el azul */
  --tone-mid: #3F6CA8;
  --tone-slate: #5B6B7A;
  --tone-sage: #6B7A5B;

  /* estados (siguen siendo monocromos, sólo variación de gris) */
  --state-bg: #E5E2D8;
  --state-ink: #111110;

  /* sombras (sutil, papel) */
  --shadow-sm: 0 1px 0 rgba(17,17,16,0.04);
  --shadow-md: 0 1px 2px rgba(17,17,16,0.04), 0 0 0 1px var(--rule);

  /* radios – geometría arquitectónica */
  --r-xs: 2px;
  --r-sm: 4px;
  --r-md: 6px;
  --r-lg: 10px;

  /* foco */
  --focus-ring: 0 0 0 3px rgba(27,58,111,0.22);

  /* alertas (mantenemos grises, sólo variación) */
  --danger: #7A1B1B;
  --danger-soft: #F1E6E2;
  --success: #2D3F2E;
  --success-soft: #E7E9DF;
  --warning: #5C4A1B;
  --warning-soft: #F1ECDB;
}

[data-theme="dark"] {
  --bg-canvas: #0E0E0D;
  --bg-surface: #141413;
  --bg-elevated: #1A1A18;
  --bg-sunken: #1F1F1D;

  --ink-1: #EFEDE5;
  --ink-2: #C5C2B8;

```

```

--ink-3:      #8A867B;
--ink-4:      #5C5A52;
--ink-5:      #3A3934;

--rule:       #26251F;      /* casi más oscuro: cohesionado con el fondo */
--rule-strong: #3A3933;
--rule-bold:  #EFEDE5;

--accent:     #7AA0D9;
--accent-deep: #A8C2E2;
--accent-soft: #1A2740;
--accent-text: #0E0E0D;
--ink-button: #EFEDE5;

--tone-mid:   #A8C2E2;
--tone-slate: #8A95A8;
--tone-sage:  #9AAA85;

--state-bg:   #2A2924;
--state-ink:  #EFEDE5;

--shadow-sm:  0 1px 0 rgba(0,0,0,0.3);
--shadow-md:  0 1px 2px rgba(0,0,0,0.4), 0 0 0 1px var(--rule);

--focus-ring: 0 0 0 3px rgba(122,160,217,0.25);

--danger:     #D8917F;
--danger-soft: #2A1816;
--success:    #A8BB95;
--success-soft: #1A1F16;
--warning:    #D6BB7A;
--warning-soft: #211D11;
}

/* === 2. RESET & BASE === */

*,
*::before,
*::after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html {
  -webkit-text-size-adjust: 100%;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-rendering: optimizeLegibility;
}

body {
  font-family: 'Maven Pro', system-ui, -apple-system, 'Helvetica Neue', sans-serif;
  font-size: 15.5px;
  line-height: 1.6;
  color: var(--ink-1);
  background-color: var(--bg-canvas);
  transition: background-color 0.25s, color 0.25s;
  min-height: 100vh;
}

/* Sutil textura de papel en el fondo */
body::before {
  content: "";
  position: fixed;
  inset: 0;

```

```

    pointer-events: none;
    z-index: 0;
    opacity: 0.35;
    background-image: radial-gradient(rgba(17,17,16,0.025) 1px, transparent 1px);
    background-size: 22px 22px;
}

[data-theme="dark"] body::before {
    background-image: radial-gradient(rgba(239,237,229,0.025) 1px, transparent 1px);
}

main, nav, section, header {
    position: relative;
    z-index: 1;
}

@keyframes fadeUp {
    from { opacity: 0; transform: translateY(8px); }
    to { opacity: 1; transform: translateY(0); }
}

.container {
    max-width: 1640px;
    margin: 0 auto;
    padding: 2.5rem 2.5rem 4rem;
    animation: fadeUp 0.4s cubic-bezier(.2,.7,.2,1);
}

/* === 3. TIPOGRAFÍA === */

h1, h2, h3, h4 {
    font-family: 'Maven Pro', system-ui, sans-serif;
    color: var(--ink-1);
    letter-spacing: -0.015em;
    line-height: 1.2;
}

h1 { font-size: clamp(2rem, 2.2vw + 1rem, 2.6rem); font-weight: 700; }
h2 { font-size: 1.5rem; font-weight: 600; }
h3 { font-size: 1.1rem; font-weight: 600; }
h4 { font-size: 0.95rem; font-weight: 600; }

a {
    color: var(--ink-1);
    text-decoration: none;
    transition: color 0.15s;
}

a:hover { color: var(--ink-2); }

p { color: var(--ink-2); }

/* utilidades tipográficas */
.mono {
    font-family: 'JetBrains Mono', 'SF Mono', Menlo, monospace;
    font-feature-settings: "calt", "ss03";
    letter-spacing: -0.01em;
    font-size: 0.9em;
}

.text-muted {
    color: var(--ink-3);
    font-weight: 400;
}

```

```

/* Scrollbar */
::-webkit-scrollbar { width: 10px; height: 10px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb {
  background: var(--rule);
  border: 2px solid var(--bg-canvas);
  border-radius: 999px;
}
::-webkit-scrollbar-thumb:hover { background: var(--rule-strong); }

/* === 4. NAVBAR === */

.navbar {
  background: var(--bg-canvas);
  border-bottom: 1px solid var(--rule);
  position: sticky;
  top: 0;
  z-index: 100;
  backdrop-filter: blur(8px);
  background-color: color-mix(in srgb, var(--bg-canvas) 92%, transparent);
}

.nav-container {
  max-width: 1640px;
  margin: 0 auto;
  padding: 0 2.5rem;
  height: 68px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1.5rem;
}

.nav-logo {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  font-weight: 600;
}

.nav-logo:hover { color: var(--ink-1); }

.logo-img {
  height: 24px;
  width: auto;
}

.logo-dark { display: none; }
[data-theme="dark"] .logo-light { display: none; }
[data-theme="dark"] .logo-dark { display: inline; }

.nav-links {
  display: flex;
  list-style: none;
  gap: 0.25rem;
  margin: 0 auto;
}

.nav-links a {
  position: relative;
  padding: 0.55rem 0.95rem;
  font-size: 0.92rem;
  font-weight: 500;
  color: var(--ink-3);
  border-radius: var(--r-sm);
  transition: color 0.15s, background-color 0.15s;
}

```



```
.nav-links a:hover {
  color: var(--ink-1);
  background-color: var(--bg-sunken);
}

.nav-links a.active {
  color: var(--ink-1);
}

.nav-links a.active::after {
  content: "";
  position: absolute;
  left: 0.95rem;
  right: 0.95rem;
  bottom: -21px;
  height: 2px;
  background: var(--accent);
}

.nav-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-width: 20px;
  height: 20px;
  padding: 0 6px;
  margin-left: 6px;
  border-radius: 9999px;
  background: var(--accent);
  color: var(--accent-text);
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.7rem;
  font-weight: 600;
  line-height: 1;
}

.nav-user-name {
  font-size: 0.9rem;
  color: var(--ink-2);
  font-weight: 500;
}

.nav-user {
  display: flex;
  align-items: center;
  gap: 0.85rem;
  padding-left: 1rem;
  border-left: 1px solid var(--rule);
}

.theme-toggle {
  background: none;
  border: 1px solid var(--rule);
  border-radius: var(--r-sm);
  padding: 0.35rem 0.5rem;
  cursor: pointer;
  color: var(--ink-2);
  display: inline-flex;
  align-items: center;
  justify-content: center;
  transition: border-color 0.15s, color 0.15s;
}

.theme-toggle:hover {
  border-color: var(--rule-strong);
  color: var(--ink-1);
}
```

```

[data-theme="light"] .icon-luna { display: none; }
[data-theme="dark"] .icon-sol { display: none; }

.hamburger {
  display: none;
  flex-direction: column;
  gap: 5px;
  background: none;
  border: none;
  cursor: pointer;
  padding: 0.5rem;
}

.hamburger span {
  width: 22px;
  height: 1.5px;
  background-color: var(--ink-1);
  border-radius: 1px;
}

/* === 5. PAGE INTRO (cabecera editorial) === */

.page-intro {
  margin-bottom: 2.5rem;
  padding-bottom: 1.75rem;
  border-bottom: 1px solid var(--rule);
}

.kicker {
  display: inline-flex;
  align-items: center;
  gap: 0.6rem;
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.78rem;
  letter-spacing: 0.14em;
  text-transform: uppercase;
  color: var(--ink-3);
  margin-bottom: 1.1rem;
}

.kicker-dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: var(--accent);
  box-shadow: 0 0 4px color-mix(in srgb, var(--accent) 15%, transparent);
  animation: pulse 2.5s ease-in-out infinite;
}

@keyframes pulse {
  0%, 100% { opacity: 1; }
  50%      { opacity: 0.35; }
}

.kicker-sep {
  opacity: 0.4;
}

.page-intro-row {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: 1.5rem;
  flex-wrap: wrap;
}

```

```

.page-title {
  font-family: 'Maven Pro', system-ui, sans-serif;
  font-weight: 700;
  font-size: clamp(2.2rem, 2.6vw + 1rem, 3rem);
  line-height: 1.05;
  letter-spacing: -0.03em;
  color: var(--ink-1);
  margin: 0;
}

.page-title .accent {
  color: var(--accent);
  font-weight: 700;
}

.page-intro-sub {
  margin-top: 0.95rem;
  font-size: 1rem;
  color: var(--ink-3);
  max-width: 64ch;
}

/* page-header simple (otras páginas) */
.page-header {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: 1.5rem;
  margin-bottom: 1.75rem;
  padding-bottom: 1.25rem;
  border-bottom: 1px solid var(--rule);
  flex-wrap: wrap;
}

.page-header h1 {
  margin-bottom: 0;
  font-size: clamp(1.8rem, 2vw + 1rem, 2.4rem);
}

/* === 5b. DASHBOARD GRID (2 columnas) === */

.dashboard-grid {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 380px;
  gap: 1.4rem;
  align-items: start;
}

.dashboard-main {
  min-width: 0; /* permite que el grid contraiga gráficos */
}

.dashboard-aside {
  position: sticky;
  top: 92px; /* navbar 68 + margen */
}

.panel-feed {
  padding: 1.6rem 1.6rem 0.6rem;
  max-height: calc(100vh - 110px);
  overflow-y: auto;
}

.feed-list {
  list-style: none;

```

```
    margin: 0;
    padding: 0;
}

.feed-item {
    border-top: 1px solid var(--rule);
}

.feed-item:first-child {
    border-top: none;
}

.feed-link {
    display: block;
    padding: 1rem 0.25rem 1.1rem;
    text-decoration: none;
    color: inherit;
    transition: background-color 0.15s;
    margin: 0 -0.25rem;
    padding-left: 0.5rem;
    padding-right: 0.5rem;
    border-radius: var(--r-xs);
}

.feed-link:hover {
    background-color: var(--bg-elevated);
    text-decoration: none;
}

.feed-meta {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 0.45rem;
}

.feed-code {
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.82rem;
    color: var(--accent);
    font-weight: 500;
    letter-spacing: -0.005em;
}

.feed-date {
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.78rem;
    color: var(--ink-3);
}

.feed-client {
    font-size: 0.98rem;
    font-weight: 600;
    color: var(--ink-1);
    line-height: 1.3;
    margin-bottom: 0.15rem;
}

.feed-device {
    font-size: 0.86rem;
    color: var(--ink-2);
    margin-bottom: 0.55rem;
}

.feed-foot {
    display: flex;
    align-items: center;
```

```

    justify-content: flex-start;
}

/* === 6. KPI CARDS (Dashboard) === */

.kpi-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 0;
  margin-bottom: 1.4rem;
  border: 1px solid var(--rule);
  border-radius: var(--r-md);
  overflow: hidden;
  background: var(--bg-surface);
}

.kpi {
  padding: 1.75rem 1.75rem 1.6rem;
  border-right: 1px solid var(--rule);
  display: flex;
  flex-direction: column;
  gap: 0.6rem;
  position: relative;
  transition: background-color 0.2s;
}

.kpi:last-child { border-right: none; }

.kpi:hover {
  background-color: var(--bg-elevated);
}

.kpi-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.kpi-label {
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.74rem;
  letter-spacing: 0.14em;
  text-transform: uppercase;
  color: var(--ink-3);
  font-weight: 500;
}

.kpi-index {
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.72rem;
  color: var(--accent);
  letter-spacing: 0.08em;
  font-weight: 500;
}

.kpi-value {
  font-family: 'Maven Pro', system-ui, sans-serif;
  font-weight: 700;
  font-size: 3rem;
  line-height: 1;
  color: var(--ink-1);
  letter-spacing: -0.035em;
  font-feature-settings: "tnum";
  margin-top: 0.45rem;
}

.kpi-value .unit {

```

```

    font-size: 1.15rem;
    color: var(--ink-3);
    margin-left: 0.3rem;
    font-weight: 500;
    letter-spacing: 0;
}

.kpi-foot {
    margin-top: 0.6rem;
    font-size: 0.86rem;
    color: var(--ink-3);
    line-height: 1.45;
}

/* === 7. PANELS (secciones del dashboard) === */

.panel {
    background: var(--bg-surface);
    border: 1px solid var(--rule);
    border-radius: var(--r-md);
    padding: 1.85rem 2rem;
    margin-bottom: 1.4rem;
    transition: background-color 0.2s;
}

.panel-head {
    display: flex;
    align-items: flex-start;
    justify-content: space-between;
    gap: 1rem;
    margin-bottom: 1.5rem;
    padding-bottom: 1rem;
    border-bottom: 1px solid var(--rule);
}

.panel-eyebrow {
    display: block;
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.74rem;
    letter-spacing: 0.14em;
    text-transform: uppercase;
    color: var(--ink-3);
    margin-bottom: 0.5rem;
    font-weight: 500;
}

.panel-eyebrow .num {
    color: var(--accent);
    font-weight: 600;
}

.panel-title {
    font-family: 'Maven Pro', system-ui, sans-serif;
    font-weight: 600;
    font-size: 1.5rem;
    line-height: 1.2;
    letter-spacing: -0.02em;
    color: var(--ink-1);
    margin: 0;
}

.panel-row {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1.25rem;
    margin-bottom: 1.25rem;
}

```

```

}

.link-arrow {
  font-size: 0.92rem;
  color: var(--accent);
  font-weight: 500;
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  border-bottom: 1px solid transparent;
  padding-bottom: 2px;
  transition: border-color 0.15s, color 0.15s;
}

.link-arrow:hover {
  color: var(--accent-deep);
  border-bottom-color: var(--accent);
}

/* === 8. PIPELINE / FLUJO OPERATIVO === */

.pipeline {
  display: grid;
  grid-template-columns: repeat(8, 1fr);
  gap: 0;
  border: 1px solid var(--rule);
  border-radius: var(--r-sm);
  overflow: hidden;
  background: var(--bg-elevated);
}

.pipeline-cell {
  padding: 1rem 0.85rem 1rem;
  border-right: 1px solid var(--rule);
  position: relative;
  display: flex;
  flex-direction: column;
  gap: 0.45rem;
  min-height: 110px;
  background: var(--bg-surface);
  transition: background-color 0.2s;
}

.pipeline-cell:last-child { border-right: none; }

.pipeline-cell:hover {
  background: var(--bg-elevated);
}

.pipeline-cell::before {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 3px;
  background: var(--ink-5);
  transition: background-color 0.2s;
}

.pipeline-cell.has-count::before {
  background: var(--accent);
}

.pipeline-step {
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.7rem;
}

```

```

    color: var(--ink-4);
    letter-spacing: 0.08em;
    font-weight: 500;
}

.pipeline-cell.has-count .pipeline-step {
    color: var(--accent);
}

.pipeline-label {
    font-size: 0.86rem;
    color: var(--ink-2);
    font-weight: 500;
    line-height: 1.3;
}

.pipeline-count {
    font-family: 'Maven Pro', system-ui, sans-serif;
    font-weight: 700;
    font-size: 2.1rem;
    line-height: 1;
    color: var(--ink-1);
    font-feature-settings: "tnum";
    margin-top: auto;
    letter-spacing: -0.025em;
}

.pipeline-cell.empty .pipeline-count {
    color: var(--ink-5);
    font-weight: 500;
}

/* === 9. CHARTS === */

.chart-wrap {
    position: relative;
    height: 290px;
}

canvas { max-width: 100%; }

/* === TABLA === */

.table-responsive {
    overflow-x: auto;
    border: 1px solid var(--rule);
    border-radius: var(--r-md);
    background: var(--bg-surface);
}

table {
    width: 100%;
    border-collapse: collapse;
    background: transparent;
}

th, td {
    padding: 0.95rem 1.25rem;
    text-align: left;
    font-size: 0.94rem;
    border-bottom: 1px solid var(--rule);
    vertical-align: middle;
}

th {
    font-family: 'JetBrains Mono', monospace;

```



```

    font-weight: 500;
    font-size: 0.72rem;
    letter-spacing: 0.14em;
    text-transform: uppercase;
    color: var(--ink-3);
    background: transparent;
    padding-top: 1.1rem;
    padding-bottom: 0.8rem;
    border-bottom-color: var(--rule-strong);
}

tr:last-child td { border-bottom: none; }

tbody tr {
    transition: background-color 0.12s;
}

tbody tr:hover {
    background: var(--bg-elevated);
}

td a {
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.88rem;
    font-weight: 500;
    color: var(--accent);
    border-bottom: 1px solid var(--rule);
    padding-bottom: 2px;
    transition: border-color 0.15s, color 0.15s;
}

td a:hover {
    border-bottom-color: var(--accent);
    color: var(--accent-deep);
}

/* === BADGES DE ESTADO === */

.badge {
    display: inline-flex;
    align-items: center;
    gap: 0.3rem;
    padding: 0.3rem 0.75rem;
    border-radius: 999px;
    font-size: 0.78rem;
    font-weight: 500;
    letter-spacing: 0.005em;
    white-space: nowrap;
    line-height: 1.3;
    border: 1px solid transparent;
}

.badge-recibido { background: #E5E2D8; color: #2F2E2A; }
.badge-diagnosticado { background: #D6D2C4; color: #2F2E2A; }
.badge-presupuesto-enviado { background: #F4E6CC; color: #6E5418; border-color: #C8A557; }
.badge-presupuesto-aceptado { background: #FBFAF6; color: var(--accent-deep); border-color: var(--accent);
font-weight: 600; }
.badge-presupuesto-aceptado::before { content: "✓"; font-size: 0.78rem; }
.badge-reparando { background: #2F2E2A; color: #FBFAF6; }
.badge-esperando-pieza { background: #2F2E2A; color: #FBFAF6; border: 1px dashed #E5E2D8; }
.badge-listo { background: var(--accent); color: #FBFAF6; font-weight: 600; }
.badge-entregado { background: var(--bg-surface); color: var(--ink-4); border-color: var(--rule); }

[data-theme="dark"] .badge-recibido { background: #2A2924; color: #EFEDE5; }
[data-theme="dark"] .badge-diagnosticado { background: #3A3933; color: #EFEDE5; }
[data-theme="dark"] .badge-presupuesto-enviado { background: #2D250F; color: #E8C77A; border-color: #8C6E20; }

```

```

[data-theme="dark"] .badge-presupuesto-aceptado { background: var(--bg-surface); color: #7AA0D9; border-color:
#7AA0D9; }
[data-theme="dark"] .badge-reparando { background: #EFEDE5; color: #0E0E0D; }
[data-theme="dark"] .badge-esperando-pieza { background: #EFEDE5; color: #0E0E0D; border: 1px dashed #5C5A52; }
[data-theme="dark"] .badge-listo { background: #7AA0D9; color: #0E0E0D; }
[data-theme="dark"] .badge-entregado { background: var(--bg-surface); color: var(--ink-4); border-color:
var(--rule); }

.badge-rol-admin {
  background: var(--ink-1);
  color: var(--accent-text);
}

.badge-rol-tecnico {
  background: var(--bg-sunken);
  color: var(--ink-2);
  border: 1px solid var(--rule);
}

/* === 12. FORMULARIOS === */

.form-group {
  margin-bottom: 1.1rem;
}

.form-group label {
  display: block;
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.75rem;
  font-weight: 500;
  color: var(--ink-3);
  margin-bottom: 0.5rem;
  text-transform: uppercase;
  letter-spacing: 0.1em;
}

.form-group input,
.form-group select,
.form-group textarea {
  width: 100%;
  padding: 0.7rem 0.9rem;
  border: 1px solid var(--rule);
  border-radius: var(--r-sm);
  font-family: inherit;
  font-size: 0.98rem;
  background-color: var(--bg-surface);
  color: var(--ink-1);
  transition: border-color 0.15s, box-shadow 0.15s;
}

.form-group input:hover,
.form-group select:hover,
.form-group textarea:hover {
  border-color: var(--rule-strong);
}

.form-group input:focus,
.form-group select:focus,
.form-group textarea:focus {
  outline: none;
  border-color: var(--accent);
  box-shadow: var(--focus-ring);
}

.form-group input::placeholder,
.form-group textarea::placeholder {
  color: var(--ink-4);
}

```

```

}

textarea {
  resize: vertical;
  min-height: 90px;
  line-height: 1.5;
}

.form-row {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 1rem;
}

.form-hint {
  font-size: 0.78rem;
  color: var(--ink-3);
  margin-top: -0.5rem;
  margin-bottom: 1.25rem;
  font-style: italic;
}

.form-actions {
  display: flex;
  gap: 0.65rem;
  margin-top: 1.5rem;
  padding-top: 1.25rem;
  border-top: 1px solid var(--rule);
}

.form-grid { display: grid; gap: 1rem; }

/* === 13. BOTONES === */

.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.45rem;
  padding: 0.62rem 1.2rem;
  border-radius: var(--r-sm);
  font-family: inherit;
  font-size: 0.92rem;
  font-weight: 500;
  cursor: pointer;
  border: 1px solid transparent;
  background: transparent;
  color: var(--ink-1);
  text-decoration: none;
  white-space: nowrap;
  transition: background-color 0.15s, border-color 0.15s, color 0.15s;
  line-height: 1.4;
}

.btn:hover { text-decoration: none; }

.btn:focus-visible {
  outline: none;
  box-shadow: var(--focus-ring);
}

.btn-primary {
  background: var(--accent);
  color: var(--accent-text);
  border-color: var(--accent);
  box-shadow: 0 1px 0 rgba(20,45,84,0.18), inset 0 1px 0 rgba(255,255,255,0.12);
}

```

```

.btn-primary:hover {
  background: var(--accent-deep);
  border-color: var(--accent-deep);
  color: var(--accent-text);
}

.btn-secondary {
  background: var(--bg-surface);
  color: var(--ink-1);
  border-color: var(--rule-strong);
}

.btn-secondary:hover {
  background: var(--bg-elevated);
  border-color: var(--ink-1);
}

.btn-lg {
  padding: 0.8rem 1.7rem;
  font-size: 1rem;
  font-weight: 600;
}

.btn-sm {
  padding: 0.42rem 0.85rem;
  font-size: 0.82rem;
}

.btn-danger {
  background: var(--danger);
  color: #FBFAF6;
  border-color: var(--danger);
}

.btn-danger:hover { opacity: 0.88; color: #FBFAF6; }

.btn-danger-text {
  color: var(--danger);
  background: transparent;
  border-color: var(--rule);
}

.btn-danger-text:hover {
  background: var(--danger);
  color: #FBFAF6;
  border-color: var(--danger);
}

/* === 14. FILTROS === */

.filtros {
  display: flex;
  flex-wrap: wrap;
  gap: 0.4rem;
  margin-bottom: 1.5rem;
  padding-bottom: 1.5rem;
  border-bottom: 1px solid var(--rule);
}

.filtros .btn {
  font-size: 0.78rem;
  padding: 0.4rem 0.95rem;
}

```

```

.filtros-avanzados {
  margin-bottom: 1.75rem;
  padding: 1rem 1.25rem;
}

.filtros-form {
  display: flex;
  flex-wrap: wrap;
  align-items: flex-end;
  gap: 0.85rem;
}

.filtros-form .form-group {
  margin-bottom: 0;
  flex: 1;
  min-width: 140px;
}

.filtros-form .form-group label {
  font-size: 0.62rem;
}

.filtros-form .form-group input,
.filtros-form .form-group select {
  padding: 0.5rem 0.7rem;
  font-size: 0.82rem;
}

.filtros-form-actions {
  display: flex;
  gap: 0.5rem;
  align-items: flex-end;
}

/* === 15. DETALLE DE REPARACIÓN === */

.detalle-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 1.25rem;
  margin: 1.5rem 0;
}

.detalle-grid .card h3 {
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.66rem;
  text-transform: uppercase;
  letter-spacing: 0.14em;
  color: var(--ink-3);
  margin-bottom: 0.85rem;
  font-weight: 500;
}

.detalle-grid .card p {
  font-size: 0.9rem;
  margin-bottom: 0.4rem;
  line-height: 1.55;
  color: var(--ink-2);
}

.detalle-grid .card p strong {
  color: var(--ink-1);
  font-weight: 500;
}

.acciones-reparacion {

```

```

    display: flex;
    flex-wrap: wrap;
    gap: 0.5rem;
    margin: 1.5rem 0;
}

.acciones-estado {
    margin: 1.75rem 0;
    padding: 1.25rem 1.5rem;
    background: var(--bg-surface);
    border: 1px solid var(--rule);
    border-radius: var(--r-md);
}

.acciones-estado h3 {
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.66rem;
    text-transform: uppercase;
    letter-spacing: 0.14em;
    color: var(--ink-3);
    margin-bottom: 0.85rem;
    font-weight: 500;
}

.acciones-estado form {
    display: inline-block;
    margin-right: 0.5rem;
    margin-bottom: 0.5rem;
}

.form-estado-inline {
    display: flex;
    align-items: flex-end;
    gap: 0.75rem;
    flex-wrap: wrap;
}

.form-estado-inline .form-group {
    margin-bottom: 0;
    min-width: 200px;
}

/* Card genérica (usada en detalles, admin, etc.) */
.card {
    background: var(--bg-surface);
    border: 1px solid var(--rule);
    border-radius: var(--r-md);
    padding: 1.5rem;
}

/* === 16. ALERTAS === */

.alert {
    padding: 0.85rem 1.1rem;
    border-radius: var(--r-sm);
    margin-bottom: 1.25rem;
    font-size: 0.86rem;
    border: 1px solid var(--rule);
    background: var(--bg-surface);
    color: var(--ink-2);
    border-left-width: 3px;
}

.alert-success {
    background: var(--success-soft);
    border-color: var(--rule);
    border-left-color: var(--success);
}

```

```

    color: var(--success);
}

.alert-error {
    background: var(--danger-soft);
    border-color: var(--rule);
    border-left-color: var(--danger);
    color: var(--danger);
}

/* === 17. MODAL === */

.modal-overlay {
    position: fixed;
    inset: 0;
    background: rgba(17,17,16,0.45);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 1000;
    backdrop-filter: blur(6px);
    animation: fadeUp 0.2s ease-out;
}

.modal {
    background: var(--bg-surface);
    border: 1px solid var(--rule);
    border-radius: var(--r-md);
    padding: 2.25rem 2rem 1.75rem;
    text-align: center;
    max-width: 420px;
    width: 90%;
    box-shadow: 0 12px 40px rgba(17,17,16,0.18);
    animation: modalIn 0.22s ease-out;
}

@keyframes modalIn {
    from { opacity: 0; transform: scale(0.97) translateY(6px); }
    to { opacity: 1; transform: scale(1) translateY(0); }
}

.modal-code {
    font-family: 'JetBrains Mono', monospace;
    font-size: 1.25rem;
    font-weight: 500;
    letter-spacing: 0;
    margin-bottom: 0.4rem;
    color: var(--ink-1);
}

.modal-msg {
    color: var(--ink-3);
    margin-bottom: 1.5rem;
    font-size: 0.88rem;
}

.modal-actions {
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
    margin-bottom: 0.85rem;
}

.modal-actions .btn {
    width: 100%;
    padding: 0.7rem 1.25rem;
    font-size: 0.88rem;
}

```

```

}

.modal-close {
  background: none;
  border: none;
  color: var(--ink-3);
  cursor: pointer;
  font-size: 0.82rem;
  padding: 0.45rem;
  font-family: inherit;
  text-decoration: underline;
  text-decoration-color: var(--rule);
  text-decoration-offset: 3px;
}

.modal-close:hover {
  color: var(--ink-1);
  text-decoration-color: var(--ink-1);
}

/* === 18. BÚSQUEDA === */

#campo-busqueda {
  font-size: 1rem !important;
  padding: 0.8rem 1.1rem 0.8rem 2.6rem !important;
  border-radius: var(--r-sm) !important;
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16'
viewBox='0 0 24 24' fill='none' stroke='%236E6B62' stroke-width='2' stroke-linecap='round'
stroke-linejoin='round'%3E%3Ccircle cx='11' cy='11' r='8'%3E%3Cline x1='21' y1='21' x2='16.65'
y2='16.65'%3E%3C/svg%3E");
  background-repeat: no-repeat;
  background-position: 0.95rem center;
}

#resultados-busqueda { margin-top: 1.25rem; }
#resultados-busqueda h2 { margin-top: 2rem; }
#resultados-busqueda p {
  padding: 2.5rem 1rem;
  text-align: center;
  color: var(--ink-3);
  font-style: italic;
  font-family: 'Fraunces', Georgia, serif;
}

/* === 19. EMPTY STATE === */

.empty-state {
  text-align: center;
  padding: 3rem 1rem;
  color: var(--ink-3);
  font-style: italic;
  font-family: 'Fraunces', Georgia, serif;
  font-size: 0.95rem;
}

/* === 20. LOGIN === */

.login-page {
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  background: var(--bg-canvas);
  padding: 1.5rem;
  position: relative;
}

```



```

.login-page::before {
  content: "";
  position: absolute;
  inset: 0;
  background-image:
    linear-gradient(var(--rule) 1px, transparent 1px),
    linear-gradient(90deg, var(--rule) 1px, transparent 1px);
  background-size: 80px 80px;
  opacity: 0.25;
  pointer-events: none;
}

.login-card {
  background: var(--bg-surface);
  border: 1px solid var(--rule);
  border-radius: var(--r-md);
  padding: 2.75rem 2.25rem 2rem;
  width: 100%;
  max-width: 400px;
  position: relative;
  z-index: 1;
}

.login-logo {
  text-align: center;
  margin-bottom: 1.75rem;
}

.login-logo .logo-img { height: 30px; }

.login-card h1 {
  text-align: center;
  font-family: 'Fraunces', Georgia, serif;
  font-weight: 380;
  font-size: 1.4rem;
  margin-bottom: 0.35rem;
  letter-spacing: -0.02em;
}

.login-sub {
  text-align: center;
  color: var(--ink-3);
  font-size: 0.85rem;
  margin-bottom: 1.75rem;
  font-style: italic;
  font-family: 'Fraunces', Georgia, serif;
}

.login-card .btn-lg { margin-top: 0.5rem; width: 100%; }

.login-theme-toggle {
  position: fixed;
  bottom: 1.5rem;
  right: 1.5rem;
  z-index: 10;
}

/* === 21. DASHBOARD HEADER (botón nueva entrada) === */

.btn-nueva-entrada {
  align-self: flex-end;
}

/* === 22. ADMIN PANEL === */

.admin-grid {

```

```

display: grid;
grid-template-columns: 1fr 2fr;
gap: 1.5rem;
margin-top: 1.5rem;
}

.admin-grid .card h3 {
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.7rem;
  text-transform: uppercase;
  letter-spacing: 0.14em;
  color: var(--ink-3);
  margin-bottom: 1.1rem;
  font-weight: 500;
}

.form-inline { display: inline; }

/* === 23. PAGINACIÓN === */

.paginacion {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 1.25rem;
  margin-top: 1.5rem;
  padding-top: 1.25rem;
}

.paginacion-info {
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.78rem;
  color: var(--ink-3);
  letter-spacing: 0.04em;
}

/* === 24. RESPONSIVE === */

@media (max-width: 1380px) {
  .dashboard-grid { grid-template-columns: minmax(0, 1fr) 340px; }
}

@media (max-width: 1180px) {
  .dashboard-grid {
    grid-template-columns: 1fr;
  }
  .dashboard-aside {
    position: static;
  }
  .panel-feed {
    max-height: none;
  }
  .feed-list {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
    gap: 0;
  }
  .feed-item {
    border-top: 1px solid var(--rule);
  }
}

@media (max-width: 1024px) {
  .kpi-grid { grid-template-columns: repeat(2, 1fr); }
  .kpi { border-right: 1px solid var(--rule); }
  .kpi:nth-child(2n) { border-right: none; }
  .kpi:nth-child(1), .kpi:nth-child(2) { border-bottom: 1px solid var(--rule); }
}

```

```

.pipeline { grid-template-columns: repeat(4, 1fr); }
.pipeline-cell:nth-child(4n) { border-right: none; }
.pipeline-cell:nth-child(-n+4) { border-bottom: 1px solid var(--rule); }
}

@media (max-width: 768px) {
  .container { padding: 1.75rem 1.25rem 3rem; }

  .hamburger { display: flex; }

  .nav-links {
    display: none;
    position: absolute;
    top: 64px;
    left: 0;
    right: 0;
    background: var(--bg-surface);
    flex-direction: column;
    padding: 0.75rem;
    border-bottom: 1px solid var(--rule);
    gap: 0.15rem;
    margin: 0;
  }

  .nav-links.active { display: flex; }

  .nav-links a {
    padding: 0.7rem 1rem;
  }

  .nav-links a.active::after { display: none; }

  .nav-user {
    margin-left: 0;
    padding-left: 0;
    border-left: none;
  }

  .panel-row { grid-template-columns: 1fr; }

  .page-intro-row { flex-direction: column; align-items: flex-start; gap: 1rem; }
  .btn-nueva-entrada { align-self: stretch; }
  .btn-nueva-entrada { width: 100%; }

  .filtros-form { flex-direction: column; }
  .filtros-form .form-group { min-width: 100%; }
  .filtros-form-actions { width: 100%; }
  .filtros-form-actions .btn { flex: 1; }

  .admin-grid { grid-template-columns: 1fr; }
  .detalle-grid { grid-template-columns: 1fr; }
}

@media (max-width: 540px) {
  .kpi-grid { grid-template-columns: 1fr; }
  .kpi { border-right: none; border-bottom: 1px solid var(--rule); }
  .kpi:last-child { border-bottom: none; }

  .pipeline { grid-template-columns: repeat(2, 1fr); }
  .pipeline-cell:nth-child(4n) { border-right: 1px solid var(--rule); }
  .pipeline-cell:nth-child(2n) { border-right: none; }

  h1 { font-size: 1.6rem; }
}

```

```
.page-title { font-size: 2rem; }  
}
```

A.6.2. estaticos/js/main.js

```
const themeToggle = document.getElementById('theme-toggle');  
const html = document.documentElement;  
  
if (themeToggle) {  
  themeToggle.addEventListener('click', () => {  
    const current = html.getAttribute('data-theme');  
    const next = current === 'light' ? 'dark' : 'light';  
    html.setAttribute('data-theme', next);  
    localStorage.setItem('theme', next);  
  });  
}  
  
const hamburger = document.getElementById('hamburger');  
const navLinks = document.getElementById('nav-links');  
  
if (hamburger) {  
  hamburger.addEventListener('click', () => {  
    navLinks.classList.toggle('active');  
  });  
  
  document.querySelectorAll('.nav-links a').forEach(link => {  
    link.addEventListener('click', () => {  
      navLinks.classList.remove('active');  
    });  
  });  
}  
  
const formNuevaEntrada = document.querySelector('.form-nueva-entrada');  
if (formNuevaEntrada) {  
  formNuevaEntrada.addEventListener('submit', (e) => {  
    const telefono = document.getElementById('telefono').value.trim();  
    const email = document.getElementById('email').value.trim();  
  
    if (!telefono && !email) {  
      e.preventDefault();  
      alert('Indica al menos un dato de contacto (teléfono o email).');  
    }  
  });  
}  
  
const telefonoInput = document.getElementById('telefono');  
const nombreInput = document.getElementById('nombre');  
const emailInput = document.getElementById('email');  
  
if (telefonoInput) {  
  let timeout;  
  telefonoInput.addEventListener('input', () => {  
    clearTimeout(timeout);  
    const q = telefonoInput.value.trim();  
    if (q.length < 3) return;  
  
    timeout = setTimeout(() => {  
      fetch(`/api/buscar-cliente?q=${encodeURIComponent(q)}`)  
        .then(res => res.json())  
        .then(data => {  
          if (data.length > 0) {  
            const c = data[0];  
            if (nombreInput) nombreInput.value = c.nombre;  
            if (emailInput && c.email) emailInput.value = c.email;  
          }  
        })  
    }, 500);  
  });  
}
```

```
    }, 300);  
});  
}
```